



UNIVERSIDAD AUTÓNOMA DEL ESTADO MÉXICO

FACULTAD DE CIENCIAS

ALGORITMOS DE ENVENENAMIENTO PARA
MODELOS DE MÁQUINAS DE VECTOR
SOPORTE

T E S I S

QUE PARA OBTENER EL TÍTULO DE:

MATEMÁTICO

PRESENTA:

AXEL MAYOLO MARTÍNEZ GONZÁLEZ

TUTOR:

MTRO. MIGUEL ANGEL LOPEZ DÍAZ

Toluca, Estado de México, 2025



Agradecimientos

Para el ciclo escolar 2022-2023, la Secretaría de Educación Pública registró 3.1 millones de estudiantes matriculados en educación superior, sin contar universidades privadas ni instituciones de formación docente. En el mismo periodo, más de 473 mil personas egresaron de alguna licenciatura o posgrado, de las cuales solo el 4.6 % pertenecía a las áreas de Ciencias Naturales, Matemáticas y Estadísticas, consideradas estratégicas para el desarrollo de la sociedad mexicana. En particular, en la carrera de Matemáticas solo egresaron 1,600 personas en los niveles de licenciatura, técnico superior universitario, maestría y doctorado, lo que representa apenas el 0.337 % del total de egresados.

Por ello, como el límite de una función creciente que tiende al infinito, agradezco a mis padres y hermanos por su apoyo incondicional, su cariño y esfuerzo, que han sido fundamentales en mi trayectoria académica.

También expreso mi gratitud a mi tutor por su dedicación, guía y consejos, los cuales serán de gran utilidad en mi futuro profesional y personal. Asimismo, agradezco a mis docentes por su enseñanza a lo largo de mi formación, particularmente a la doctora Rocío, por motivarme a continuar en esta increíble área de la ciencia.

De igual manera, agradezco a mis compañeros, de quienes aprendí innumerables cosas, por las horas compartidas, los proyectos realizados y las experiencias vividas. En especial, a Isaac, Alex, Carlos, Joaquín, Edgar Y Víctor, quienes hicieron de este camino una experiencia inefable.

Finalmente, extendiendo mi gratitud a todas las personas que, de alguna manera, han influido en mi formación tanto personal como académica, con la esperanza de que, con el tiempo, el número de egresados en Matemáticas siga en aumento.

Notación

Introducción

En este trabajo se examina en profundidad el funcionamiento de las máquinas de soporte vectorial (SVM), un algoritmo de aprendizaje automático concebido por Vladimir Vapnik y su equipo en los laboratorios de AT&T durante la década de 1960 y presentado en su libro *The nature of statistical learning theory*, en 1999. Aunque originalmente diseñado para abordar problemas de clasificación binaria, el modelo ha evolucionado gracias a investigaciones que han extendido su alcance a la clasificación multiclase y aplicaciones en problemas de regresión.

El capítulo 1 está enfocado principalmente en describir intuitivamente todos los conceptos necesarios para entender la construcción del modelo SVM. En este apartado se describen de manera clara y concisa los elementos clave, como la separación óptima de clases, el margen de decisión y los vectores de soporte. Asimismo, se abordan otros temas que pueden obviarse en un principio, pero que son necesarios para comprender por completo el modelo.

La Sección 2 desarrolla el marco matemático formal del modelo SVM, comenzando con la formulación del problema de margen duro. Posteriormente, se aborda su generalización al problema de margen blando, que permite manejar datos no separables linealmente. Además, se introduce el concepto de transformación mediante funciones kernel, lo que posibilita la extensión del modelo a espacios de características de mayor dimensionalidad.

El capítulo 3 reinterpreta el problema SVM desde la perspectiva de la regularización, destacando cómo esta formulación alternativa se relaciona con el problema de margen blando. Además, se muestra la equivalencia entre ambas metodologías.

El capítulo 4 se enfoca en un análisis detallado del procedimiento conocido como

envenenamiento. Esta es una técnica adversarial en la que se inyectan datos estratégicamente diseñados para comprometer la precisión del modelo. Este capítulo explora los fundamentos teóricos de este tipo de ataque.

Finalmente, los dos últimos capítulos están dedicados a aplicaciones prácticas. En ellos se implementa el modelo SVM utilizando herramientas de software para analizar el conjunto de datos MNIST. Posteriormente, se incluyen las conclusiones derivadas del análisis, sintetizando los hallazgos clave del trabajo y sugiriendo estrategias para la defensa de los modelos ante el envenenamiento.

Índice general

Agradecimientos	I
Notación	II
Introducción	III
1 Conceptos Basicos	1
§1.1 Introducción: Motivación	1
§1.2 Máquinas de soporte vectorial (Idea intuitiva)	3
§1.3 Envenamiento (Idea intuitiva)	4
§1.4 Conceptos basicos de Geometría	6
§1.5 Teoria de la optimización	9
§1.6 Conceptos basicos de Algebra	12
§1.7 Espacios de Hilbert con nucleo Reproducible	13
2 Hiperplanos separadores	18
§2.1 Caso Separable. Hiperplano de Margen Maximo	18
§2.2 Hiperplano de Margen Maximo Caso No Separables	27
§2.3 Máquinas de Soporte Vectorial	32
3 Penalización y Nucleos Reproducibles	38
§3.1 SVM como metodo de penalización	38
§3.2 SVM en terminos de RKHS	40
4 Envenamiento de SVM	46
§4.1 Envenenamiento	46

5	Aplicaciones practicas	55
§5.1	Aplicaciones: Digits	56
§5.1.1	Envenenamiento	59
§5.2	Aplicaciones: Wine	65
§5.2.1	Envenenamiento	66
6	Metodos de defensa y conclusiones	73
§6.1	Metodos de defensa	74
§6.1.1	Detección de Valores Atípicos	74
§6.1.2	Procedencia de Datos	75
§6.1.3	Modelo De-Pois	76
§6.1.4	Modelo TRIM	77
§6.2	Conclusiones	80
	Apéndice A	81
	Apéndice B	83

Capítulo 1

Conceptos Basicos

1.1. Introducción: Motivación

El *aprendizaje estadístico* siempre ha jugado un rol importante en diferentes áreas de la ciencia, la finanza y la industria, y más recientemente en nuevas áreas como lo son la minería de datos, el *machine learning* y la inteligencia artificial. Pero para poder obtener conclusiones del aprendizaje estadístico, es necesario que deduzcamos una función capaz de predecir un valor deseado a partir de algunos datos, ejemplos o entradas, a menudo llamados *inputs*, aunque también algunos autores les llaman predictores o variables independientes. Estos *inputs* pueden provenir esencialmente de medidas cuantitativas (salario, precio de un producto, índice de grasa, etc.), categóricas (Posee la enfermedad/No posee la enfermedad) o una combinación de ambas. En cualquier caso, nuestro interés se centra en que, a partir de los *inputs*, podamos construir un modelo de predicción que nos permitirá decidir el resultado de nuevos objetos que no se encuentran en el conjunto de datos. A este problema se le llama *aprendizaje supervisado*, debido a que se caracteriza por usar conjuntos de datos etiquetados para entrenar los algoritmos que clasifican nuevos datos (en el aprendizaje no supervisado, los algoritmos se entrenan con datos sin etiquetar y usualmente se utilizan para descubrir patrones y relaciones entre datos).

A los valores predichos a partir de nuestro modelo de predicción se les llama valores de salida, variables de respuesta, variables dependientes o, más comúnmente en la

literatura, **outputs**, y al igual que en los *inputs*, los *outputs* pueden tomar valores tanto cualitativos como cuantitativos.

Al conjunto de mediciones de *inputs* y *outputs* se le llama **datos de entrenamiento**, y nos servirán para construir el modelo predictivo. En la variable predictora (*outputs*) es importante distinguir las variables cualitativas (también denominadas variables categóricas, discretas o de factores) de las cuantitativas. Esta distinción en los *outputs* ha llevado a una nomenclatura para la tarea de predicción:

- Regresión, cuando los *outputs* son medidas cuantitativas.
- Clasificación, cuando los *outputs* son variables cualitativas.

Como notación, usaremos x como una variable de entrada. Notemos que esta variable puede ser un vector, donde las coordenadas correspondientes del vector se representan mediante subíndices, mientras que el i -ésimo valor observado de x se escribe como x_i . Las variables de salida se notarán por y , en el caso de las variables cualitativas algunos autores usan la letra g (grupo), en el presente documento usaremos la letra y .

Para un grupo de dos clases, el conjunto de *outputs* tomará valores $\{0, 1\}$ o $\{-1, 1\}$, según sea el caso (en las máquinas de soporte vectorial, usualmente utilizamos el conjunto $\{-1, 1\}$). Dado que todos los vectores se asumen como vectores columna, la fila i -ésima de x es x_i^T , la transpuesta del vector x_i .

La tarea de aprendizaje se expresa de la siguiente manera: dada una entrada representada por el vector x , debemos realizar una buena predicción de la salida y , que denotamos como $\hat{y}$ (pronunciado “y-tilde”), mediante alguna función. Si y toma valores en $\mathbb{R}$, entonces $\hat{y}$ también debería hacerlo; lo mismo ocurre para salidas categóricas g , donde $\hat{G}$ debería tomar valores en el mismo conjunto G . Note que es posible transformar el grupo G en un conjunto de números mediante una biyección en subconjuntos de $\mathbb{N}$, en la que a cada elemento de G se le asocia un único elemento de este subconjunto.

1.2. Máquinas de soporte vectorial (Idea intuitiva)

En el presente trabajo desarrollaremos el modelo de *máquinas de soporte vectorial* (*SVM*, por sus siglas en inglés), el cual pertenece a la categoría de modelos de clasificación. La idea esencial de este modelo consiste en lo siguiente: dado un conjunto de *inputs*, donde cada uno pertenece a una de dos categorías, se construye un modelo capaz de predecir si un nuevo *input* pertenece a una categoría o a la otra. Usualmente, estos *inputs* se representan como puntos en diagramas de dispersión para facilitar su visualización.

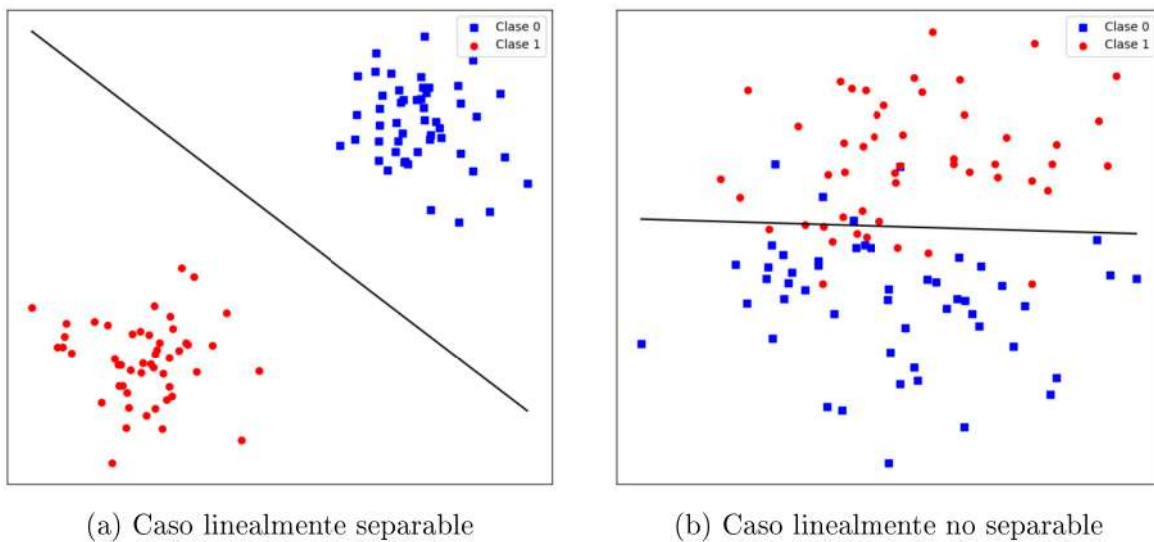


Figura 1.1: En ambas imágenes se presenta un hiperplano, pero en la primera es posible construir un hiperplano que separe correctamente todas las entradas, mientras que en la segunda no hay forma de construirlo, debido a la distribución de los datos.

El *SVM* busca un hiperplano que separe de forma óptima los puntos de una clase de los de la otra. Es importante aclarar que no en todos los conjuntos de datos es posible encontrar un hiperplano que logre una separación perfecta; sin embargo, podemos construir un modelo óptimo que minimice la cantidad de datos mal clasificados (ver Figura 1.1). En los casos en los que sí es posible encontrar un hiperplano que separe correctamente todos los datos, los denominamos casos separables. Por el contrario, si no es posible encontrar un hiperplano que separe correctamente todos los datos, los denominaremos casos no separables. Cabe señalar que dicho hiperplano puede no ser

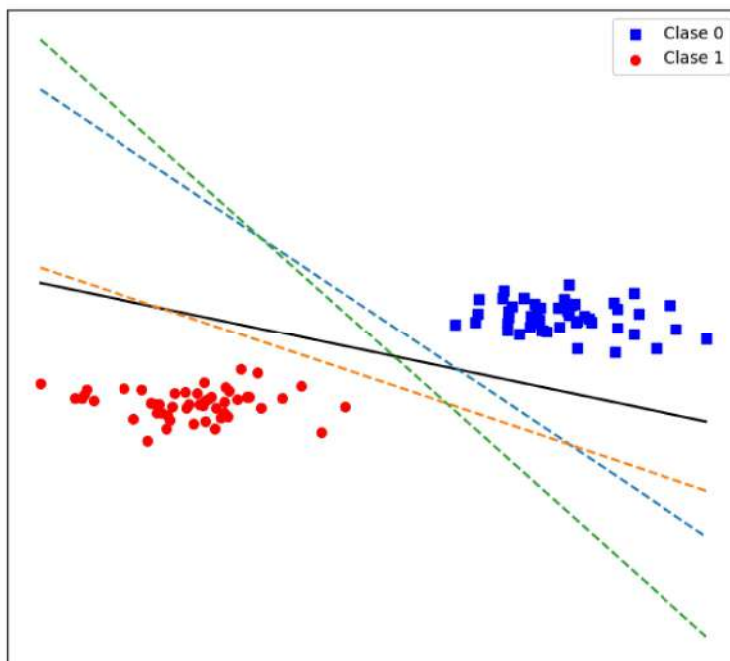


Figura 1.2: Pueden existir multiples hiperplanos como los representados por las lineas punteadas, pero nuestro problema radica en encontrar el que maximiza la distancia del hiperplano a los puntos más cercanos de cada clase.

único por lo que buscamos el hiperplano que maximice la distancia con los puntos más cercanos a él (ver figura 1.2), por este motivo las *SVM* también son conocidas como clasificadores de margen máximo. A estos puntos que se encuentran más cerca del hiperplano, pero que a su vez maximiza la distancia se les suele llamar *vectores de soporte* (hablaremos de ellos con más detalle en la Sección 2).

1.3. Envenenamiento (Idea intuitiva)

El aprendizaje estadístico puede ser usado como una herramienta en la toma de decisiones en una gran variedad de problemas, incluyendo aquellos relacionados con la seguridad, pues un ejemplo de su aplicación puede ser la identificación de comportamientos irregulares. En este ámbito, se ha utilizado para la detección de spam, intrusiones o fraudes. Sin embargo, recientemente han surgido problemas relacionados con el envenenamiento de datos. Este es un tipo de ataque en el que se manipula o contamina el conjunto de datos utilizado en el entrenamiento de los modelos de apren-

dizaje automático, con el fin de comprometer la precisión y eficacia de los algoritmos y los procesos de toma de decisiones.

Por ejemplo, algunas empresas pueden modificar sus estrategias para evitar ser detectadas por los filtros de spam, lo que genera la necesidad de desarrollar métodos de aprendizaje que sean robustos ante este tipo de datos envenenados. Los ataques contra los algoritmos de aprendizaje automático pueden clasificarse en varios tipos, pero nos centraremos principalmente en los ataques de envenenamiento, los cuales pertenecen a una categoría denominada ataques causales, en los que se inyectan puntos de datos especialmente diseñados para corromper el proceso de aprendizaje. Este tipo de ataque es particularmente relevante desde el punto de vista práctico, ya que, aunque generalmente no se tiene acceso directo a la base de datos de entrenamiento, es posible introducir nuevos datos en el conjunto de entrenamiento (por ejemplo, cuando los datos son recopilados a través de Internet), lo que brinda una oportunidad para envenenar los datos.

En particular, nos centraremos en una familia específica de ataques de envenenamiento contra las (*SVM*). Estos ataques consisten en inyectar datos en el conjunto de entrenamiento que han sido diseñados específicamente para aumentar el error de prueba del modelo en la función de pérdida. En el peor de los casos, se asume que el atacante conoce el algoritmo de aprendizaje y puede extraer datos de la misma distribución o incluso conocer los datos de entrenamiento utilizados por el modelo (a esto se le conoce como modelo de caja blanca o *white-box*). Si asumimos que tenemos una solución óptima al problema de entrenamiento de una *SVM*, es posible construir un modelo de ataque insertando puntos diseñados específicamente para afectar una medida de rendimiento del modelo (generalmente la penalización por error). Esto puede formularse como un problema de optimización, donde el objetivo es maximizar el impacto en la medida de rendimiento, manteniendo al mismo tiempo la solución óptima del problema de entrenamiento.

Para identificar los puntos de ataque, se puede utilizar el procedimiento de ascenso del gradiente, que busca máximos locales en la superficie del error de prueba, aunque esta superficie sea generalmente no convexa. Este método tiene la ventaja (desde el

punto de vista del atacante) de que solo depende de los gradientes de los productos punto en el espacio de entrada, por lo que puede ser kernelizado (lo que sea que esto signifique por el momento, en secciones posteriores definiremos estos conceptos) lo que nos permite trabajar en espacios de características de alta dimensión sin necesidad de transformar explícitamente los datos a ese espacio. Esto abre nuevas posibilidades para optimizar los ataques basados en datos contra algoritmos que utilizan kernels, lo que podría hacer que dichos algoritmos sean más vulnerables a ataques de envenenamiento.

Antes de desarrollar el modelo, haremos un breve resumen de algunos resultados útiles para su construcción. Sin embargo, al ser un tema que extiende muchos conceptos, no indagaremos en los detalles, pero dejaremos referencias para consulta.

1.4. Conceptos basicos de Geometría

El modelo lineal es el método más usual para describir la relación entre una variable dependiente y una o más variables independientes. Dado un vector de entradas

$$x^T = (x_1, x_2, \dots, x_p)$$

y un vector de coeficientes

$$\hat{\beta} = (\beta_1, \beta_2, \dots, \beta_p)$$

podemos predecir un valor de salida Y mediante el modelo lineal:

$$\hat{Y} = \hat{\beta}_0 + \sum_{j=1}^p x_j \hat{\beta}_j$$

Donde β_0 es el Sesgo¹ o también conocido como *Intercepto* en el aprendizaje automático.

En el caso particular de los problemas de clasificación usamos este modelo lineal como una forma de describir el hiperplano que nos divida correctamente a los datos.

¹Sesgo: En el contexto de los modelos lineales el Sesgo nos dice el valor promedio esperado para la variable de respuesta y , cuando las variables predictoras x_i son iguales a cero.

Matemáticamente, un hiperplano para el caso p -dimensional son todos los puntos que satisfacen la ecuación

$$\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p = 0 \quad (1.1)$$

Naturalmente, si un punto satisface la ecuación anterior, entonces dicho punto está en el hiperplano. En caso de que un punto X no satisfaga la ecuación (1.1), pueden ocurrir dos casos:

$$\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p = \beta_0 + \sum_{j=1}^p X_j \hat{\beta}_j > 0 \quad (1.2)$$

$$\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p = \beta_0 + \sum_{j=1}^p X_j \hat{\beta}_j < 0 \quad (1.3)$$

Esto nos dice que la entrada x se encuentra en algún lado del hiperplano, pero no sobre este. Por lo que podemos pensar en el hiperplano como una división del espacio p -dimensional en dos mitades. Para ver en qué lado del plano se encuentra una observación X , basta con calcular el signo de la ecuación (1.1).

En lo consiguiente para un mejor entendimiento daremos unas propiedades geométricas en $\mathbb{R}^2$, pero que facilmente podrian ser generalizadas a $\mathbb{R}^n$. Como mencionanemos anteriormente la siguiente ecuación representa un hiperplano o conjunto afín L definido por

$$f(x) = \beta_0 + \beta^T x = 0$$

dado que estamos en $\mathbb{R}^2$, esto es una línea recta.

Para cualquier dos puntos x_1 y x_2 que se encuentren en L ,

$$\beta^T (x_1 - x_2) = 0$$

pues ambos puntos satisfacen la ecuación del hiperplano y por lo tanto el producto punto de la diferencia de estos dos puntos con el vector β^T es cero, es decir,

$$\beta^* = \frac{\beta}{\|\beta\|}$$

es el vector normal unitario a la superficie de L .

Para cualquier punto x_0 en L ,

$$\beta^T x_0 = -\beta_0$$

La distancia firmada (esto es el valor de las distancias pero que dependen del signo de acuerdo al lado del hiperplano que se encuentren) de cualquier punto x a L está dada por

$$\beta^{*T}(x - x_0) = \frac{1}{\|\beta\|}(\beta^T x + \beta_0) = \frac{1}{\|f'(x)\|}f(x)$$

Esto se puede deducir de las propiedades anteriores y del hecho de que $f'(x) = \beta$.

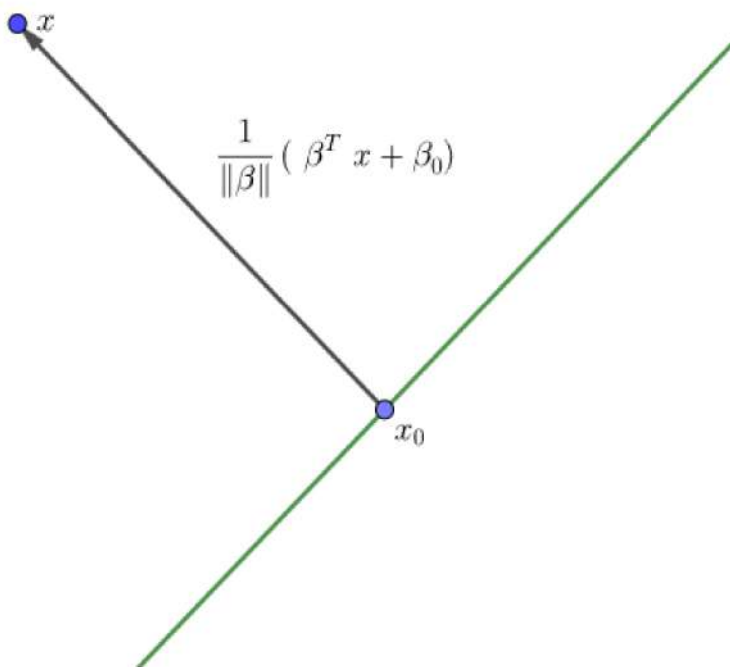


Figura 1.3: Distancia de un punto al hiperplano

Esta propiedad nos será de gran utilidad más adelante, cuando intentemos identificar cuáles son los vectores con la menor distancia al hiperplano.

Nuestro objetivo, entonces, es encontrar este hiperplano que separe los datos maximizando esta distancia a los puntos de cada clase. Para ello, existen diferentes métodos de construcción, los cuales desarrollaremos con más detalle en el siguiente capítulo.

1.5. Teoría de la optimización

Para resolver un problema de Machine Learning es necesario resolver un problema de optimización, en el caso de las SVM se tiene que resolver un problema convexo con restricciones lineales, por lo que en el presente capítulo daremos un breve resumen a la teoría de optimización

Un problema de optimización está compuesto por una función objetivo $f(x)$ y algunas restricciones $c_i(x)$, con $i = 1, \dots, n$. Esencialmente, el problema de optimización tiene el siguiente esquema (La función objetivo puede ser tanto para maximizar como para minimizar).

$$\begin{aligned} \text{mín} \quad & f(x) \\ \text{Sujeto a } & c_j(x) \leq 0 \end{aligned} \tag{1.4}$$

A menudo se reformula el problema de optimización introduciendo la función lagrangiana:

$$L(x, \alpha) = f(x) - \sum_{i=1}^n \alpha_i c_i(x) \tag{1.5}$$

donde $f(x)$ es la función objetivo, a los α_i se les conocen como los multiplicadores de Lagrange, y $c_i(x)$ son las restricciones asociadas a los α_i . Esta función lagrangiana nos relaciona la función objetivo con las restricciones en una sola función, que nos ayudara para resolver el problema.

Sin embargo a veces el problema puede ser lo suficientemente complejo como para buscar alternativas, por lo que en la teoría de optimización suele aparecer el concepto de *dual*. El objetivo es proporcionar una formulación alternativa de un problema de optimización que esté relacionada con el problema principal. Al problema original se le conoce como *primal*, y al problema transformado, como el problema *dual*. La teoría de la optimización asociada con un problema de programación convexa destaca que, si el problema primal no es convexo, entonces el problema dual puede no tener una solución

a partir de la cual podamos obtener otra solución para el problema primal. Por esta razón, en lo que sigue, consideraremos únicamente funciones convexas.

Una forma particular de estas transformaciones, que son convenientes desde el punto de vista computacional, es el llamado Dual Wolfe. El fin del Dual del Wolfe es relacionar las condiciones de primer orden, remplazando las condiciones de restricción por un requisito de optimalidad en la función lagrangiana. R. Fletcher, (1981) describe las condiciones de primer orden, estas son conocidas comúnmente como condiciones de Karush-Kuhn-Tucker (KKT).

Por ejemplo, supongamos que tenemos un problema de optimización de la forma:

$$\begin{aligned} \text{mín} \quad & c^T x \\ \text{Sujeto a } & A^T x + b \leq 0 \end{aligned} \tag{1.6}$$

donde $c, x \in \mathbb{R}^n$, $d \in \mathbb{R}^n$, y $A \in \mathbb{R}^{n \times m}$, además $Ax + d \leq 0$ es la forma matricial de escribir $\sum_{j=1}^m A_{ij}x_j + d_i \leq 0$. Podemos usar el dual de Wolfe, ya que la función y las restricciones en (1.6) implican que es un problema de optimización convexo, por lo que la teoría de optimización nos asegura que el problema primal se convierte en:

$$\begin{aligned} \text{máx} \quad & d^T \alpha \\ \text{Sujeto a } & A^T = c\alpha \\ & \alpha \geq 0 \end{aligned} \tag{1.7}$$

Es importante observar que el número de variables y restricciones ha cambiado, ya que, en principio teníamos m variables y n restricciones, ahora tenemos n variables junto con m restricciones de igualdad y n restricciones de desigualdad. La dualización nos ayudará a que los problemas sean más sencillos de resolver, aunque también puede ocurrir lo contrario.

Los siguientes dos teoremas muestran la relación entre las soluciones del problema

primal y el dual.

Teorema 1 *Si x^* resuelve el problema primal de programación convexa, y si f y c_i , $i = 1, 2, \dots, m$, son funciones C^1 , y si se cumple la suposición de regularidad, entonces x^*, α^* resuelve el problema dual*

$$\begin{aligned} \text{mín} \quad & L(x, \alpha) \\ \text{sujeto a} \quad & \nabla_x L(x, \alpha) = 0, \quad \alpha \geq 0 \end{aligned}$$

Además, los valores mínimos de la función primal y los máximos de la función dual son iguales, es decir, $f^* = L(x^*, \alpha^*)$.

En otras palabras, este teorema nos indica que, si tenemos un problema de programación convexa, entonces, bajo las suposiciones de regularidad (Condiciones de KKT), resolver el problema dual nos da la misma solución óptima que resolver el problema primal, y existen multiplicadores de Lagrange que hacen esto posible.

Condiciones de Karush-Kuhn-Tucker (KKT) *Si x^* es un minimizador local del problema y si las condiciones de regularidad se cumple en x^* , entonces existen multiplicadores de Lagrange α^* tales que x^* y α^* satisfacen el siguiente sistema de ecuaciones:*

$$\begin{aligned} \nabla_x L(x, \alpha) &= 0 \\ c_i(x) &\geq 0 \\ \alpha_i &\geq 0 \\ \alpha_i c_i(x) &= 0 \quad \forall i \end{aligned} \tag{1.8}$$

Describir las condiciones de regularidad van fuera de los propósitos de este trabajo pero se pueden consultar en *R. Fitcher*, 1981, p. 55. El interés en las condiciones de Karush-Kuhn-Tucker (KKT) radica en que establece las condiciones que han de cumplirse para poder resolver el problema primal utilizando el dual, mientras que el Teorema 1 nos asegura que resolver el problema dual nos da las mismas soluciones que el primal. En resumen, partiendo del problema primal se construye la función lagran-

giana. Seguidamente, se aplican las condiciones del teorema KKT y se obtienen un conjunto de relaciones que, al ser sustituidas en la función lagrangiana, hacen desaparecer todas las variables primales, y la función dual obtenida dependerá únicamente de los multiplicadores de Lagrange. Por lo tanto, la solución al problema primal se obtiene sustituyendo el resultado obtenido al resolver el problema dual.

1.6. Conceptos basicos de Algebra

En muchas ocasiones, no es posible construir un modelo lineal que separe los datos de manera adecuada. Por lo tanto, es necesario ir más allá de la linealidad. La idea es aumentar el vector de entradas X con más variables, que son transformaciones de X , y luego usar modelos lineales en este espacio ampliado. Este enfoque permite crear modelos lineales en el espacio ampliado, aunque sean modelos no lineales en el espacio original, lo que nos ayuda proporcionando mayor flexibilidad y capacidad para capturar relaciones complejas en los datos.

Antes de continuar, daremos unas definiciones importantes descritas por Hofmann et al., 2008.

Definición. Un **kernel** es una función $K : X \times X \rightarrow \mathbb{R}$ que mide la similitud entre dos puntos en un espacio con producto interno. La función kernel se define de tal manera que, para todos $x, x' \in X$,

$$K(x, x') = \langle h(x), h(x') \rangle_{\mathcal{H}}, \quad (1.9)$$

donde $\langle \cdot, \cdot \rangle$ denota el producto punto en el espacio de características (entiendase de momento como un espacio ampliado) y $h : X \rightarrow \mathcal{H}$ es una función que mapea el espacio de entrada X en un espacio de características $\mathcal{H}$ con producto escalar. El kernel permite calcular el producto punto en el espacio de características $\mathcal{H}$ sin necesidad de mapear explícitamente los puntos a ese espacio.

Definición. Dado un kernel K y entradas $x_1, \dots, x_n \in X$, la matriz $n \times n$

$$\mathbf{K} := (k(x_i, x_j))_{ij}$$

se llama la **matriz de Gram** (o matriz de kernel) de K con respecto a $x_1, \dots, x_n$.

Definición. Sea X un conjunto no vacío. Una función $K : X \times X \rightarrow \mathbb{R}$ que para todo $n \in \mathbb{N}$, $x_i \in X$, $i = 1, \dots, n$ da lugar a una matriz de Gram que cumple que

$$\sum_{i,j=1}^n c_i c_j \mathbf{K}_{ij} \geq 0 \quad (1.10)$$

para todo $c_i \in \mathbb{R}$ es llamada **kernel definida positiva**.

En resumen los kernels, pueden considerarse como productos punto generalizados, que nos dan una medida de similitud entre los puntos en el espacio $\mathcal{H}$. Usualmente los mas usados dentro del contexto de las máquinas de soporte vectorial son, el kernel polinomial, la función de base radial (RBF) y kernel sigmoideal, en el capítulo 3 daremos mas detalles de su uso.

1.7. Espacios de Hilbert con nucleo Reproducible

Como se mencionó en la sección anterior, en ocasiones no es posible construir un modelo lineal en el espacio de entradas, por lo que se tiene que ampliar la dimensión a un espacio de características para poder construir un modelo de clasificación lineal en este espacio. Los espacios de Hilbert con Kernel reproducible (RKHS) nos ayudan con esta tarea, como se verá en esta sección.

La mayoría de los modelos de Machine Learning permiten una representación como función de regularización.

La regularización es un método que nos permite restringir el proceso de estimación para evitar un sobreajuste del modelo. Una clase general de problemas de regularización tiene el siguiente esquema:

$$\min_{f \in \mathcal{H}} \left[\sum_{i=1}^N L(y_i, f(x_i)) + \lambda J(f) \right] \quad (1.11)$$

Donde:

- $L(y_i, f(x_i))$ es una función de pérdida.

- $J(f)$ es una función de penalización.
- $\mathcal{H}$ es un espacio de funciones sobre el cual $J(f)$ esta definido.

Bajo ciertas suposiciones se puede demostrar que las soluciones son de la forma

$$f(X) = \sum_{k=1}^N \alpha_k \phi_k(X) + \sum_{i=1}^M \theta_i G(X - x_i) \quad (1.12)$$

Donde ϕ_k abarcan el espacio nulo de la funcional de penalización J , y G es la transformada inversa de Fourier de alguna función $\tilde{G}$. Es importante destacar que, aunque el criterio (1.11) se define sobre un espacio de dimensión infinita, las soluciones se caracterizan por tener dimensión finita.

Una subclase importante de problemas del tipo (1.11) son los generados por las funciones kernels $K(x, y)$ y el correspondiente espacio de funciones $\mathcal{H}_K$, al que se le llama *Espacio de Hilbert con Kernel Reproducible*.

La teoría de los espacios de Hilbert con núcleo reproducible nos asegura que, para cada función kernel definida positiva, es posible construir su espacio de Hilbert con núcleo reproductor asociado y viceversa, Rohan Shiloh, 2007. En particular, tenemos el teorema de Moore-Aronszajn que nos dice: *Todo kernel definido positivo $K(\cdot, \cdot)$ en $X \times X$ es un kernel reproductor para algún RKHS único de funciones en X . Y a la inversa, todo RKHS tiene un kernel definido positivo asociado*. Más adelante veremos que las máquinas de soporte vectorial pueden determinar el hiperplano óptimo en este espacio asociado utilizando el llamado **truco del kernel**.

Sean $x, y \in \mathbb{R}^p$ y consideremos el espacio de funciones generado por $\{K(\cdot, y) \mid y \in \mathbb{R}^p\}$, es decir, combinaciones lineales de la forma $f(x) = \sum_m \alpha_m K(x, y_m)$. Donde los términos del kernel se ven como una función de primer argumento y está indexado por el segundo. Además supongamos que K tiene una expansión en valores propios de la forma

$$K(x, y) = \sum_{i=1}^{\infty} \gamma_i \phi_i(x) \phi_i(y) \quad (1.13)$$

con $\gamma_i \geq 0$, $\sum_{i=1}^{\infty} \gamma_i^2 < \infty$. Los elementos de $\mathcal{H}_K$ se pueden ver como una expansión

en términos de funciones propias

$$f(x) = \sum_{i=1}^{\infty} c_i \phi_i(x) \quad (1.14)$$

Con la restricción

$$\|f\|_{\mathcal{H}_K}^2 \stackrel{\text{def}}{=} \sum_{i=1}^{\infty} \frac{c_i^2}{\gamma_i} < \infty \quad (1.15)$$

Donde $\|f\|_{\mathcal{H}_K}^2$ es una norma inducida por K . La función de penalización en (1.11) en el espacio $\mathcal{H}_K$ se define como $J(f) = \|f\|_{\mathcal{H}_K}^2$. $J(f)$ se puede ver como una penalización donde las funciones con grandes eigen valores se penalizan menos y viceversa. Podemos reescribir (1.11) como

$$\min_{f \in \mathcal{H}_K} \sum_{i=1}^N L(y_i f(x_i)) + \lambda \|f\|_{\mathcal{H}_K}^2 \quad (1.16)$$

o equivalentemente

$$\min_{\{c_j\}_{j=1}^{\infty}} \left[\sum_{i=1}^N L \left(y_i, \sum_{j=1}^{\infty} c_j \phi_j(x_i) \right) + \lambda \sum_{j=1}^{\infty} c_j^2 \gamma_j \right] \quad (1.17)$$

Se puede demostrar que la solución de (1.16) tiene dimensión finita y de la forma

$$f(x) = \sum_{i=1}^N \alpha_i K(x, x_i) \quad (1.18)$$

La función $h_i(x) = K(x, x_i)$ es una función base (Como función del primer argumento) tambien conocida como el *representador de evaluación* en x_i en $\mathcal{H}_K$. Se puede probar que para $f \in \mathcal{H}_K$ $\langle K(\cdot, x_i), f \rangle_{\mathcal{H}_K} = f(x_i)$. Analogamente $\langle K(\cdot, x_i), K(\cdot, x_j) \rangle_{\mathcal{H}_K} = K(x_i, x_j)$ (la propiedad de reproducción de $\mathcal{H}_K$, esta propiedad nos muestra que es posible remplazar el producto de vectores en el espacio de características con una evaluación del núcleo sobre el espacio de entrada y a esto se le suele llamar truco *truco del kernel*) y por tanto

$$J(f) = \sum_{i=1}^N \sum_{j=1}^N K(x_i, x_j) \alpha_i \alpha_j \quad (1.19)$$

Para $f(x) = \sum_{i=1}^N \alpha_i K(x, x_i)$. Usando estas dos ultimas ecuaciones en (1.16), obtene-

mos el criterio de dimensión finita

$$\min_{\alpha} [L(y, \mathbf{K}\alpha) + \lambda \alpha^T \mathbf{K}\alpha] \quad (1.20)$$

Donde $\mathbf{K}$ es una matriz de $N \times N$ con entradas $K(x_i, x_j)$. Así un criterio de dimensión infinita, se reduce aun problema de dimensión finita, a esto se le denomina la propiedad del nucleo.

En el capítulo 3, una vez que se defina la máquina de soporte vectorial, podremos observar que el problema de clasificación de dos clases tiene la forma

$$f(x) = \alpha_0 + \sum_{i=1}^N \alpha_i K(x, x_i) \quad (1.21)$$

donde los parámetros minimizan la función

$$\min_{\alpha_0, \alpha} \left[\sum_{i=1}^N (1 - y_i f(x_i))_+ + \frac{\lambda}{2} \alpha^T \mathbf{K}\alpha \right] \quad (1.22)$$

Esta representación del modelo corresponde a un problema de regularización, como el descrito en la ecuación 1.11.

Donde $[z]_+$ denota la función de pérdida, conocida como *hinge loss*, que mapea cada variable en un número real. Intuitivamente, representa el error o costo de las variables al ser clasificadas. La función de pérdida nos ayuda a determinar cómo se penalizan los errores en los problemas de clasificación y regresión, en consecuencia, afecta el proceso de optimización del modelo.

Esta función también suele presentarse como

$$\ell^{\text{hinge}}(t, y) = \begin{cases} 0 & \text{si } yt \geq 1 \\ 1 - yt & \text{en otro caso} \end{cases} \quad (1.23)$$

Nótese que cuando la distancia de un punto es mayor que +1, el valor de la pérdida es 0. En cambio, si la distancia es menor que 1, la función incurre en una pérdida; a una distancia de 0, la pérdida es de +1. Los datos clasificados correctamente tendrán

una pérdida nula, mientras que las instancias clasificadas incorrectamente tendrán una pérdida mayor. De esta forma, hinge loss es un indicador de la cantidad de puntos mal clasificados, mientras que si la pérdida es cero, indica que los puntos están correctamente clasificados.

El uso de la función Hinge loss en SVM permite encontrar un hiperplano óptimo que maximiza el margen entre clases. Sin embargo, su naturaleza no diferenciable en ciertos puntos ha llevado al desarrollo de versiones suavizadas, como la variante de Huber, que facilitan la optimización mediante métodos de descenso de gradiente.

Cabe destacar que esta función es convexa, lo que facilita el uso de métodos de optimización como el descenso de gradiente, y aunque no sea diferenciable, se puede resolver usando subgradientes. También existen versiones suavizadas que pueden ser preferibles a la hora de la optimización, por ejemplo, las propuestas de Rennie y Srebro (2005) o de Rosset y Zhu (2007), llamada Huberised Square Hinge Loss.

Por otra parte, podríamos usar la ecuación (1.22) para resolver el problema de SVM; sin embargo, daremos una construcción geométrica y, en el capítulo 3, detallamos la equivalencia entre estas dos formas de abordar el problema.

Capítulo 2

Hiperplanos separadores

Como mencionamos anteriormente, nuestro objetivo reside en buscar un hiperplano que separe correctamente a los datos. A los clasificadores que buscan una combinación lineal de las características de entrada y devuelven el signo se les llaman perceptrones (Rosenblatt, 1948), que son clasificadores lineales y el precursor de las redes neuronales. Desde el punto de vista geométrico, el perceptrón busca trazar un límite de decisión lineal en el espacio de características, con el cual poder decidir la clasificación de las observaciones. Sin embargo, Vapnik, en 1996, junto con los laboratorios de AT&T, desarrollaron un nuevo método denominado Máquinas de Soporte Vectorial, que describiremos en esta sección. Para este problema pueden ocurrir dos casos: en el primero, el conjunto es linealmente separable, es decir, existe un hiperplano separador que clasifica correctamente a todos los datos de entrenamiento; en el segundo caso, el conjunto no es linealmente separable, por lo que se pueden permitir algunos términos clasificados en el lado incorrecto del margen. Estos son conocidos como hiperplano de margen máximo e hiperplano de margen suave, respectivamente (hard margin and soft margin), ver figura 1.1.

2.1. Caso Separable. Hiperplano de Margen Maximo

Dado un conjunto de entrenamiento $(x_1, y_1, x_2, y_2, \dots, x_N, y_N)$, donde $x_i \in \mathbb{R}^p$ y $y_i \in \{-1, +1\}$, podemos construir el *hiperplano de margen máximo*, el cual se construye

calculando la distancia (perpendicular) desde cada observación de ambas clases a un hiperplano separador dado.

A la distancia mínima de todas las observaciones al hiperplano se le conoce como *margen*.

El hiperplano de margen máximo es el hiperplano separador para el cual el margen es más grande, es decir, es el hiperplano que tiene la mayor distancia al punto más cercano de los datos de entrenamiento.

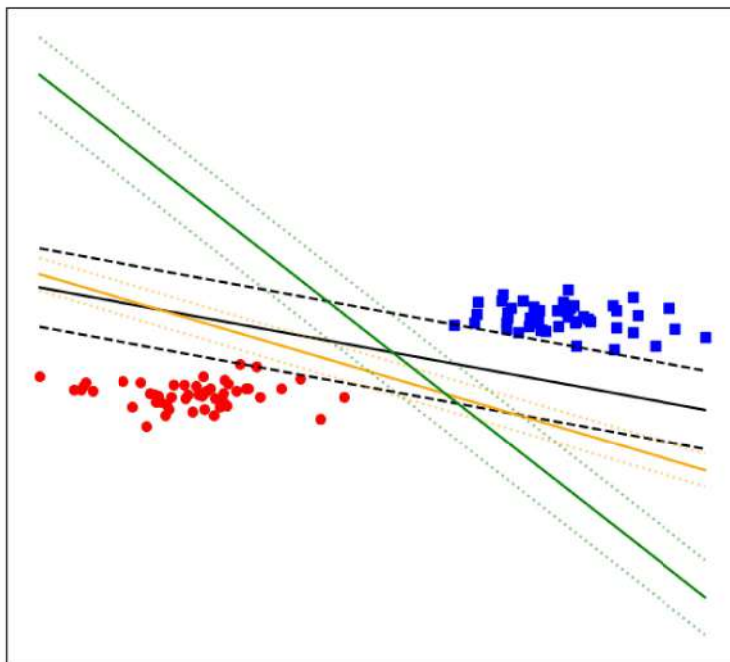


Figura 2.1: Las líneas continuas muestran algunos de los posibles hiperplanos de separación, y las líneas punteadas, amarillas y verdes, muestran sus márgenes, en el caso de la recta negra y sus correspondientes margen es la solución óptima que maximiza el tamaño del margen, y al que llamamos hiperplano de margen máximo.

Este hiperplano que separa los datos, a veces también se le conoce como *el clasificador de margen máximo*. Para saber cómo se clasifica una observación de prueba X^* , nos basamos en el signo de

$$f(x^*) = \beta_0 + \beta_1 x_1^* + \beta_2 x_2^* + \cdots + \beta_p x_p^* = \beta_0 + \sum_{i=1}^p \beta_i x_i^*$$

donde $\beta_0, \dots, \beta_p$ son los coeficientes del hiperplano de margen máximo. Los puntos

para los cuales la distancia del plano a estos (de tal forma que el margen sea el mayor) es la mínima se les conoce como *vectores de soporte* y son vectores en un espacio p -dimensional que 'soportan' el hiperplano de margen máximo, en relación con que si estos puntos se movieran ligeramente, el hiperplano también se movería.

Para un primer caso, consideremos que los datos son separables, es decir, que es posible encontrar un hiperplano que divida a los datos, de tal forma que todas las observaciones sean clasificadas correctamente.

El modelo matemático del hiperplano de margen máximo se construye a continuación. Un hiperplano que separe correctamente debe satisfacer las siguientes dos condiciones:

$$\beta_0 + x_i^T \beta \geq 0 \quad \text{si } y_i = +1 \quad (2.1)$$

$$\beta_0 + x_i^T \beta \leq 0 \quad \text{si } y_i = -1 \quad (2.2)$$

Es otras palabras, cada observación la clasifica en algun lado del plano según el signo de la función.

O bien, podemos reescribir las condiciones como:

$$y_i(\beta_0 + x_i^T \beta) \geq 0 \quad \text{para todo } i = 1, \dots, N \quad (2.3)$$

Luego la distancia firmada de un punto x_i , (el valor positivo de la distancia que depende del lado del hiperplano en el que se encuentre) al hiperplano es

$$\frac{x_i^T \beta + \beta_0}{\|\beta\|} \quad (2.4)$$

(ver capítulo 1.4). Sea M es el margen descrito en la parte anterior. Haciendo uso de (2.37), la ecuación anterior y del hecho de que cada punto debe de estar a una distancia mayor que la del margen, entonces tenemos

$$\frac{1}{\|\beta\|} (y_i(x_i^T \beta + \beta_0)) \geq M \quad (2.5)$$

es importante destacar que esta desigualdad se da ya que buscamos que los puntos esten al menos a una distancia mayor que el ancho del margen. Ahora notemos que podemos reescribir la ecuación anterior como

$$y_i(x_i^T \beta + \beta_0) \geq M \|\beta\|$$

Así tenemos el problema de optimización:

$$\max_{\beta, \beta_0} M \quad \text{sujeto a} \quad y_i(x_i^T \beta + \beta_0) \geq M \|\beta\| \quad \text{para } i = 1, \dots, N \quad (2.6)$$

En el que buscamos maximizar el margen M sujeto a que la distancia de cualquier observación al hiperplano sea mayor o igual que el tamaño del margen.

Si consideramos β y β_0 tales que satisfacen estas desigualdades, entonces cualquier múltiplo positivo también las satisface, por lo que, por convenio, se toma $\|\beta\| = \frac{1}{M}$.

Entonces, $M \cdot \|\beta\| = 1$.

Por lo que nuestro problema ahora radica en minimizar $\|\beta\|$. Con el fin de simplificar el análisis, elevamos la función al cuadrado y multiplicamos por $\frac{1}{2}$, obteniendo así una función convexa. Esta transformación no modifica la posición del mínimo de la función, así que la ecuación (2.6) es equivalente a:

$$\min_{\beta, \beta_0} \frac{1}{2} \|\beta\|^2 \quad \text{sujeto a} \quad y_i(x_i^T \beta + \beta_0) \geq 1 \quad \text{para } i = 1, \dots, N \quad (2.7)$$

Este es un problema de optimización cuadrática, con restricciones de desigualdad lineal, (ver capítulo 1.5). El cual tiene una forma dual si la función a optimizar y las restricciones son estrictamente convexas, en este caso resolver el problema dual es equivalente a obtener la solución al problema primal (se puede mostrar que tanto la función objetivo como las restricciones son funciones estrictamente convexas).

Haciendo uso de la función de Lagrange Primal con el objetivo de minimizar con

respecto a β y β_0 ,

$$\begin{aligned} L_p &= \frac{1}{2} \|\beta\|^2 - \sum_{i=1}^N \alpha_i (y_i (x_i^T \beta + \beta_0) - 1) \\ &= \frac{1}{2} \langle \beta, \beta \rangle - \sum_{i=1}^N \alpha_i y_i (x_i^T \beta) - \sum_{i=1}^N \alpha_i y_i \beta_0 + \sum_{i=1}^N \alpha_i \end{aligned} \quad (2.8)$$

Derivando parcialmente respecto de β y β_0 respectivamente tenemos

$$\begin{aligned} \frac{\partial L_p}{\partial \beta} &= \frac{1}{2} (\beta \cdot 1 + 1 \cdot \beta) - \sum_{i=1}^N \alpha_i y_i x_i \\ &= \beta - \sum_{i=1}^N \alpha_i y_i x_i \end{aligned} \quad (2.9)$$

$$\frac{\partial L_p}{\partial \beta_0} = \sum_{i=1}^N \alpha_i y_i \quad (2.10)$$

Igualando las derivadas a cero,

$$\beta = \sum_{i=1}^N \alpha_i y_i x_i \quad (2.11)$$

y

$$\sum_{i=1}^N \alpha_i y_i = 0 \quad (2.12)$$

Sustituyendo en la ecuación (2.8).

$$\begin{aligned}
\frac{1}{2}\|\beta\|^2 - \sum_{i=1}^N \alpha_i (y_i(x_i^T \beta + \beta_0) - 1) &= \frac{1}{2} \left\langle \sum_{i=1}^N \alpha_i y_i x_i, \sum_{i=1}^N \alpha_i y_i x_i \right\rangle - \sum_{i=1}^N \alpha_i y_i \left(x_i^T \sum_{k=1}^N \alpha_k y_k x_k \right) \\
&\quad - \sum_{i=1}^N \alpha_i y_i \beta_0 + \sum_{i=1}^N \alpha_i \\
&= \frac{1}{2} \sum_{i=1}^N \alpha_i y_i x_i^T \left(\sum_{k=1}^N \alpha_k y_k x_k \right) \\
&\quad - \sum_{i=1}^N \alpha_i y_i \left(x_i^T \sum_{k=1}^N \alpha_k y_k x_k \right) + \sum_{i=1}^N \alpha_i \\
&= \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{k=1}^N \alpha_i y_i x_i^T \alpha_k y_k x_k
\end{aligned} \tag{2.13}$$

Por lo que tenemos el siguiente problema

$$L_D = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{k=1}^N \alpha_i y_i x_i^T \alpha_k y_k x_k \tag{2.14}$$

Sujeto a $\alpha_i \geq 0$, también notemos que la expresión $\sum_{i=1}^N \alpha_i y_i \beta_0$ se cancela debido a que $\sum_{i=1}^N \alpha_i y_i = 0$, por lo que este término se considera como una restricción adicional. Aunque el problema original se formula en términos de una minimización, el dual emplea el objetivo opuesto. Por lo que ahora el problema radica en encontrar los multiplicadores de Lagrange que maximizan la función dual, sujeto a las restricciones anteriores. A la ecuación (2.14) se le llama el dual de Wolfe junto con las restricciones y la denotamos por L_D .

Por las condiciones de Karush–Kuhn–Tucker,

$$\alpha_i [y_i (x_i^T \beta + \beta_0) - 1] = 0 \quad \forall i \tag{2.15}$$

De esto notemos que si $\alpha_i > 0$ entonces $y_i (x_i^T \beta + \beta_0) = 1$ esto nos indica que x_i está en el espacio de separación (se encuentra en los extremos del margen), por la condición en la ecuación (2.7). A estos puntos en el extremo del margen les llamamos *vectores de soporte*.

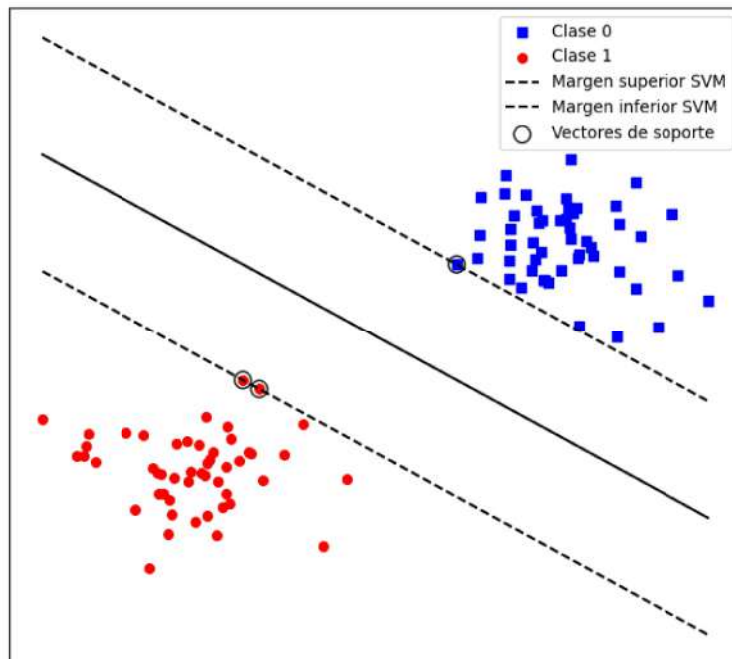


Figura 2.2: Los puntos con $\alpha_i > 0$, se llaman vectores de soporte, ya que el hiperplano depende de estos, en la imagen corresponde a los puntos encerrados en la circunferencia

Por otra parte, si $y_i(x_i^T \beta + \beta_0) - 1 > 0$, entonces x_i no está en el espacio de separación, es decir, está a una distancia mayor que los extremos del margen, por lo tanto, en la condición (2.15), $\alpha_i = 0$.

Por otra parte, observemos que podemos calcular el vector β en la expresión (2.11), ya que se define únicamente en términos de una combinación lineal de los vectores de soporte, es decir, aquellos puntos para los cuales α_i es mayor que cero.

De la misma forma, podemos obtener el valor de β_0 resolviendo la ecuación (2.15) para alguno de los vectores de soporte.

Por lo tanto, el plano de separación óptimo produce una función $\hat{f}(x) = x^T \hat{\beta} + \hat{\beta}_0$ para clasificar nuevas observaciones,

$$\hat{G}(x) = \text{sign} \hat{f}(x)$$

que depende únicamente de los vectores de soporte (pues β y β_0 dependen únicamente de ellos).

Para ilustrar mejor el funcionamiento de las máquinas de soporte vectorial, consi-

deremos un ejemplo sencillo: supongamos un conjunto de entrenamiento de dos puntos A y C , sean y_A y y_C sus respectivas etiquetas, entonces tenemos el problema

$$\min_{\beta, \beta_0} \frac{1}{2} \|\beta\|^2 \quad \text{sueto a} \quad y_A(A^T\beta + \beta_0) = 1, \quad y_C(C^T\beta + \beta_0) = 1 \quad (2.16)$$

En este caso se da la igualdad en las restricciones, pues se necesita que estos dos puntos formen el hiperplano separador. Definamos la función de lagrange como

$$L_p = \frac{1}{2} \|\beta\|^2 - \alpha_A(y_A(A^T\beta + \beta_0)) - \alpha_C(y_C(C^T\beta + \beta_0))$$

Podemos optimizar esta función mediante el metodo de lagrange. Sin perdida de generalidad supongamos que $y_A = 1$ y $y_C = -1$

$$L_p = \frac{1}{2} \|\beta\|^2 - \alpha_A(A^T\beta + \beta_0) + \alpha_C(C^T\beta + \beta_0)$$

Entonces derivando parcialmente

$$\begin{aligned} \frac{\partial L_p}{\partial \beta} &= \frac{1}{2}(\beta \cdot 1 + 1 \cdot \beta) - \alpha_A A^T + \alpha_C C^T \\ &= \beta - \alpha_A A^T + \alpha_C C^T = 0 \end{aligned}$$

de donde

$$\beta = \alpha_A A^T - \alpha_C C^T \quad (2.17)$$

Por otra parte

$$\frac{\partial L_p}{\partial \beta_0} = -\alpha_A + \alpha_C = 0$$

De esto ultimo obtenemos que $\alpha_A = \alpha_C$. Entonces sustituyendo en (2.17), obtenemos

$$\beta = \alpha_A(A^T - C^T) \quad (2.18)$$

y de las restricciones de (2.16) podemos obtener $(A^T\beta + \beta_0) = 1$, $(C^T\beta + \beta_0) = -1$, restando estas dos ecuaciones obtenemos

$$(A^T - C^T)\beta = 2 \quad (2.19)$$

luego multiplicando (2.18), por $(A^T - C^T)$

$$(A^T - C^T)\beta = (A^T - C^T)\alpha_A(A^T - C^T) = \alpha_A(A^T - C^T)^2$$

y sustituyendo (2.19) en la ecuación anterior obtenemos

$$2 = \alpha_A(A^T - C^T)^2$$

Por lo que

$$\alpha_A = \alpha_C = \frac{2}{(A^T - C^T)^2}$$

Y en consecuencia

$$\begin{aligned} \beta &= 2 \frac{(A^T - C^T)}{(A^T - C^T)^2} \\ \beta_0 &= 1 - A^T\beta = 1 - 2A^T \frac{(A^T - C^T)}{(A^T - C^T)^2} = \frac{-(A^T)^2 + (C^T)^2}{(A^T - C^T)^2} \end{aligned}$$

Así, si tenemos dos puntos A y C , podemos obtener los coeficientes del hiperplano a partir de A y C .

Cabe destacar que, aunque ninguna de las observaciones de entrenamiento caiga en el margen, por la forma en la que está construido el modelo, para las observaciones de prueba no necesariamente se cumple esto, por lo que un margen amplio en los datos de entrenamiento llevará a una buena separación de los datos de prueba.

Notamos que usamos fuertemente el hecho de que se cumpla la condición (2.1) y (2.2), es decir, que los datos sean separables. Cuando los datos no son separables, no hay una solución factible para este problema usando el modelo anterior. Por lo tanto, haremos uso de un criterio alternativo al (2.1) y (2.2).

2.2. Hiperplano de Margen Maximo Caso No Separables

Supongamos ahora que las clases no son separables, es decir, que para algunas observaciones no se cumple (2.37), por lo que hay observaciones en el lado incorrecto del plano separador. Nuestro objetivo ahora es medir qué tanto estas variables están violando la restricción y minimizar esta violación de la restricción.

Para ello, definamos las variables de holgura $\xi = (\xi_1, \xi_2, \dots, \xi_N)$, con $\xi_i \geq 0$. Podemos modificar las restricciones en (2.5) para obtener la restricción

$$y_i(x_i^T \beta + \beta_0) \geq M(1 - \xi_i) \quad (2.20)$$

para todo i , $\xi_i \geq 0$ y $\sum_{i=1}^N \xi_i \leq \text{constante}$. Esta restricción nos mide la distancia relativa al margen, es decir, la distancia de los puntos al margen dependiendo del ancho del margen (hay otros métodos, pero en particular este modelo nos lleva a un problema de optimización convexa).

ξ_i nos indica la cantidad proporcional por la cual la predicción $f(x_i) = x_i^T \beta + \beta_0$ está en el lado incorrecto de su margen. Dicho de otro modo, ξ_i mide cuánto se permite que el punto x_i viole la restricción; si $\xi_i > 1$, entonces x_i está en el lado incorrecto del hiperplano, fuera de los márgenes, mientras que si $\xi_i = 0$, entonces la observación está en el lado correcto del margen, y si $0 < \xi_i < 1$, entonces solo está en el lado incorrecto del margen, sin salirse de este. Por otra parte, al acotar $\sum_{i=1}^N \xi_i$, estamos limitando el número de malas clasificaciones en el entrenamiento. Supongamos que esta constante es C .

Como en (2.7) podemos reescribir el problema considerando $M = \frac{1}{\|\beta\|}$, como:

$$\text{mín } \|\beta\| \quad \text{sujeto a} \quad y_i(x_i^T \beta + \beta_0) \geq 1 - \xi_i \quad \forall i \quad \xi_i \geq 0 \quad \sum_{i=1}^N \xi_i \leq C \quad (2.21)$$

Esta es la forma usual en la que se nos presenta el clasificador de vectores de soporte

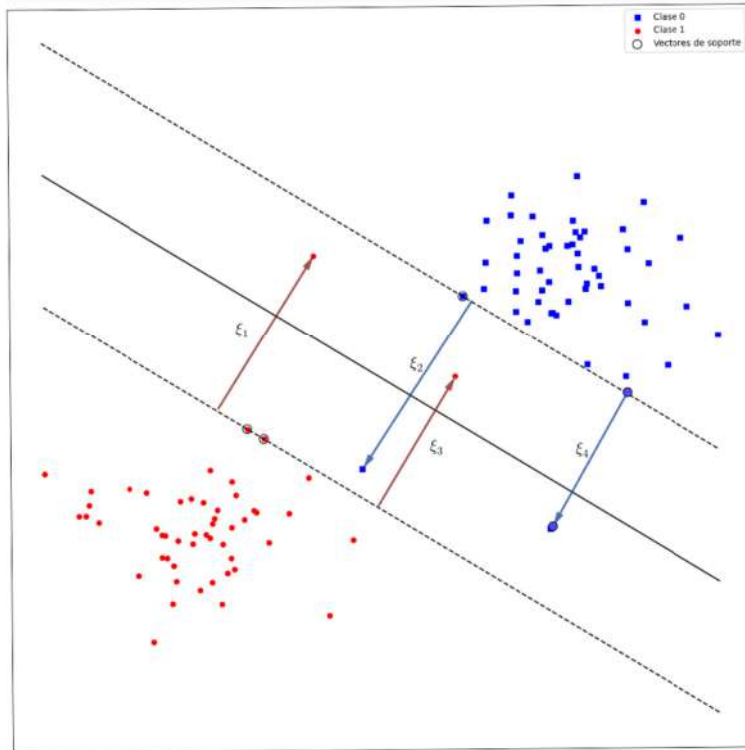


Figura 2.3: Los puntos a los cuales se les asocia un ξ_i , son puntos que se encuentran, en el lado incorrecto de su margen.

para el caso no separable. El cual vuelve a ser un problema de optimización convexa con restricciones de desigualdad lineal y podemos reescribir este problema en su forma equivalente

$$\min_{\beta, \beta_0} \quad \frac{1}{2} \|\beta\|^2 + C \sum_{i=1}^N \xi_i \quad \text{sueto a} \quad \xi_i \geq 0 \quad \text{y} \quad y_i(x_i^T \beta + \beta_0) \geq 1 - \xi_i \quad \forall i \quad (2.22)$$

Donde C representa el *costo* y este limita la suma de los ξ_i . Es decir, la suma de los ξ_i determina el número de violaciones al margen que podemos tolerar. Puesto que si ξ_i es mayor que 1, entonces la observación x_i está en el lado incorrecto del hiperplano, para cada i . Por lo tanto, $\sum_{i=1}^n \xi_i$ es menor o igual a C . Así, al considerar un costo mayor, aumenta la cantidad de tolerancia para variables mal clasificadas, y por lo tanto tendremos que considerar un margen mayor. Mientras que si tomamos un valor de C pequeño, creamos un modelo que es menos tolerante a las violaciones del margen y, por lo tanto, el margen es más estrecho. En la práctica, este valor es elegido por el usuario

y puede estimarse mediante validación cruzada.

Así, el nuevo problema de optimización consistirá en encontrar el hiperplano de separación definido por β y β_0 , que minimice (??). Al hiperplano resultante se le denomina hiperplano de separación de margen blando (Soft margin). La manera de obtener el hiperplano de separación es análoga al caso separable, por lo que los pasos siguientes son similares a los de la sección anterior.

La función de Lagrange primal es:

$$\begin{aligned} L_P &= \frac{1}{2} \|\beta\|^2 + C \sum_{i=1}^N \xi_i - \sum_{i=1}^N \alpha_i [y_i(x_i^T \beta + \beta_0) - (1 - \xi_i)] - \sum_{i=1}^N \mu_i \xi_i \\ &= \frac{1}{2} \langle \beta, \beta \rangle + C \sum_{i=1}^N \xi_i - \sum_{i=1}^N \alpha_i y_i x_i^T \beta - \sum_{i=1}^N \alpha_i y_i \beta_0 + \sum_{i=1}^N \alpha_i (1 - \xi_i) - \sum_{i=1}^N \mu_i \xi_i \end{aligned} \quad (2.23)$$

Que busquemos minimizar con respecto a β , β_0 y ξ_i . Y con α_i y μ_i multiplicadores de lagrange

Derivando parcialmente tenemos

$$\begin{aligned} \frac{\partial L_P}{\partial \beta} &= \frac{\partial}{\partial \beta} \left[\frac{1}{2} \langle \beta, \beta \rangle + C \sum_{i=1}^N \xi_i - \sum_{i=1}^N \alpha_i y_i x_i^T \beta - \sum_{i=1}^N \alpha_i y_i \beta_0 + \sum_{i=1}^N \alpha_i (1 - \xi_i) - \sum_{i=1}^N \mu_i \xi_i \right] \\ &= \beta - \sum_{i=1}^N \alpha_i y_i x_i^T \end{aligned} \quad (2.24)$$

$$\begin{aligned} \frac{\partial L_P}{\partial \beta_0} &= \frac{\partial}{\partial \beta_0} \left[\frac{1}{2} \langle \beta, \beta \rangle + C \sum_{i=1}^N \xi_i - \sum_{i=1}^N \alpha_i y_i x_i^T \beta - \sum_{i=1}^N \alpha_i y_i \beta_0 + \sum_{i=1}^N \alpha_i (1 - \xi_i) - \sum_{i=1}^N \mu_i \xi_i \right] \\ &= - \sum_{i=1}^N \alpha_i y_i \end{aligned} \quad (2.25)$$

$$\begin{aligned} \frac{\partial L_P}{\partial \xi_i} &= \frac{\partial}{\partial \xi_i} \left[\frac{1}{2} \langle \beta, \beta \rangle + C \sum_{i=1}^N \xi_i - \sum_{i=1}^N \alpha_i y_i x_i^T \beta - \sum_{i=1}^N \alpha_i y_i \beta_0 + \sum_{i=1}^N \alpha_i - \sum_{i=1}^N \alpha_i \xi_i - \sum_{i=1}^N \mu_i \xi_i \right] \\ &= C - \alpha_i - \mu_i \end{aligned} \quad (2.26)$$

Igualando a cero las ecuaciones 2.24-2.26 obtenemos

$$\beta = \sum_{i=1}^N \alpha_i y_i x_i^T \quad (2.27)$$

$$0 = \sum_{i=1}^N \alpha_i y_i \quad (2.28)$$

$$C = \alpha_i + \mu_i \quad (2.29)$$

Sustituyendo en en la función L_p obtenemos la función dual L_D

$$\begin{aligned} L_D &= \frac{1}{2} \left\langle \sum_{i=1}^N \alpha_i y_i x_i, \sum_{k=1}^N \alpha_k y_k x_k^T \right\rangle + \sum_{i=1}^N C \xi_i - \sum_{i=1}^N \alpha_i y_i x_i \sum_{i=1}^N \alpha_k y_k x_k^T \\ &\quad - \beta_0 \sum_{i=1}^N \alpha_i y_i + \sum_{i=1}^N \alpha_i (1 - \xi_i) - \sum_{i=1}^N \mu_i \xi_i \\ &= \frac{1}{2} \sum_{i=1}^N \sum_{k=1}^N \alpha_i y_i x_i \alpha_k y_k x_k^T + \sum_{i=1}^N (\alpha_i + \mu_i) \xi_i - \sum_{i=1}^N \sum_{i=1}^N \alpha_k \alpha_i y_i x_i y_k x_k^T + \sum_{i=1}^N \alpha_i \\ &\quad - \sum_{i=1}^N \alpha_i \xi_i - \sum_{i=1}^N \mu_i \xi_i \\ &= \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{k=1}^N \alpha_i y_i x_i \alpha_k y_k x_k^T \end{aligned} \quad (2.30)$$

Por lo que buscamos maximizar L_D sujeto a $0 \leq \alpha_i \leq C$ y $\sum_{i=1}^N \alpha_i y_i = 0$, ademas de que las condiciones de Karush-Kunh-Tucker incluyen

$$\alpha_i [y_i (x_i^T \beta + \beta_0) - (1 - \xi_i)] = 0 \quad (2.31)$$

$$\mu_i \xi_i = 0 \quad (2.32)$$

$$y_i (x_i^T \beta + \beta_0) - (1 - \xi_i) \geq 0 \quad (2.33)$$

De la ecuación 2.29, si tomamos $\alpha_i = 0$, entonces $C = \mu_i$, por lo que en la ecuación

2.32 se deduce que $\xi_i = 0$. Se concluye entonces que para las entradas x_i cuyo $\alpha_i = 0$, corresponden a casos separables.

En los casos no separables, x_i se caracteriza por $\xi_i > 0$, por lo que nuevamente de (2.32), $\mu_i = 0$ y por 2.29 se concluye que $C = \alpha_i$. Así, cuando $C = \alpha_i$, corresponde a casos no separables.

Ahora supongamos que $0 < \alpha_i < C$. De la condición (2.29), deducimos que $\mu_i \neq \alpha_i$, y de esto, y usando la restricción (2.32), obtenemos que $\xi_i = 0$. Por otra parte, como $0 < \alpha_i < C$ de la restricción (2.31), y de lo deducido anteriormente ($\xi_i = 0$), se concluye que

$$y_i (x_i^T \beta + \beta_0) - 1 = 0.$$

Es decir, x_i está en la frontera del margen. Y por lo tanto, x_i es un vector de soporte si y solo si $0 < \alpha_i < C$.

Notemos que de la ecuación (2.34):

$$\hat{\beta} = \sum_{i=1}^N \hat{\alpha}_i y_i x_i^T \quad (2.34)$$

$\hat{\beta}$ es una combinación lineal de los $\hat{\alpha}_i$ no nulos, para los cuales x_i cumple las restricciones (2.31) - (2.33), en otras palabras $\hat{\beta}$ queda determinado únicamente por los vectores de soporte y de las observaciones para los cuales $\alpha_i = C$.

Y para obtener el valor de β_0 usamos la expresión (2.31), para verlo como solución de alguno de los vectores de soporte (para los cuales $0 < \hat{\alpha}$ y $\hat{\xi}_i = 0$), aunque usualmente se ocupa un promedio de todas estas soluciones.

Maximizar el dual es un problema de programación cuadrática convexa, mas facil de resolver que el primal.

Una vez obtenidas β_0 y β , podemos escribir la función de decisión dada por

$$\hat{G}(x) = \text{sign} [\hat{f}(x)] = \text{sign} [x^T \hat{\beta} + \hat{\beta}_0] \quad (2.35)$$

Notemos que la unica diferencia entre ambos modelos del caso separable y del caso no separable es el parametro de ajuste C .

2.3. Máquinas de Soporte Vectorial

Uno de los principales problemas al abordar aplicaciones con SVM es que estas pueden no ser separables linealmente y permitir múltiples errores, lo que puede llevarnos a un modelo con baja precisión. Por lo tanto, en este capítulo nos centraremos en extender la idea desarrollada en el capítulo anterior, para ampliar el espacio de variables de entrada y construir un límite de decisión lineal entre las variables de entrada y las de salida, en un espacio ampliado. Es importante destacar que el espacio ampliado puede ser muy grande e incluso infinito. Sin embargo, como mencionaremos más adelante, la solución al problema solo dependerá de los vectores de soporte.

La *máquina de vectores de soporte con kernel* nos permite ampliar el espacio de variables de entrada utilizado por el clasificador de vectores de soporte, lo que nos conducirá a un clasificador lineal en el espacio ampliado, pero que en el espacio original no es lineal (ver figura 3.1). Llamaremos *espacio de entradas* al espacio original y *espacio de características* al espacio ampliado $\mathcal{H}$, en principio solo requerimos que $\mathcal{H}$ tenga definido un producto punto.

Sea $h(x_i) : X \rightarrow \mathcal{H}$ una función que mapea nuestro espacio de entradas al espacio de características. Donde $h(x_i) = (h_1(x_i), h_2(x_i), \dots, h_M(x_i))$ para $i = 1, \dots, N$ y $h_m(x_i)$ para $m = 1, \dots, M$ estas últimas son llamadas las funciones base. Entonces podemos representar el problema de optimización (2.23) a través de los vectores de entrada y la función $h(x_i)$ mediante el producto interno. Por lo que la función dual de lagrange (2.30) en el espacio de características es

$$L_D = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{i'=1}^N \alpha_i \alpha_{i'} y_i y_{i'} \langle h(x_i), h(x_{i'}) \rangle \quad (2.36)$$

Y la solución $f(x)$ puede reescribirse como:

$$f(x) = h(x)^T \beta + \beta_0 = \sum_{i=1}^N \hat{\alpha}_i y_i \langle h(x), h(x_i) \rangle + \beta_0 \quad (2.37)$$

Sin embargo para realizar esto se requiere el calculo de del producto interno $\langle h(x_i), h(x_{i'}) \rangle$

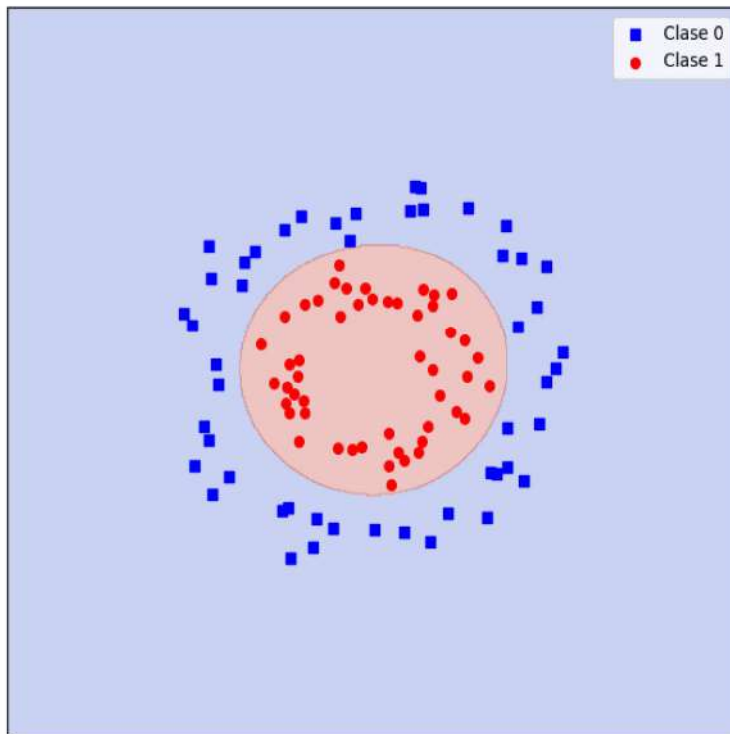


Figura 2.4: No hay forma de separar los datos usando un hiperplano lineal, por lo que se amplía el espacio de entrada, para construir un modelo lineal en el espacio de características, en este caso se ha usado el kernel base radial.

en un espacio de alta dimension, afortunadamente el uso de un kernel definido positivo K puede reducir significativamente estos calculos complejos

de modo que $K(x, x') = \langle h(x), h(x') \rangle$ por lo que podemos reescribir (2.37) como

$$f(x) = \sum_{i=1}^N \hat{\alpha}_i y_i K(x, x_i) + \hat{\beta}_0$$

pedimos que k sea definido positivo, por que nos proporciona un problema convexo que puede resolverse de manera eficiente. Es posible considerar una clase más amplia de kernels sin alterar la convexidad del problema cuadrático. Esto se debe a que las restricciones del problema de optimización excluyen ciertas regiones del espacio de multiplicadores α_i . En consecuencia, solo requerimos que el kernel sea definido positivo en los puntos relevantes, es decir, condicionalmente definido positivo.

Comunmente en las aplicaciones sobre SVM, se ocupan 3 tipos de Kernels

- *Kernel polinómico de grado d*

$$K(x, x') = (1 + \langle x, x' \rangle)^d \quad (2.38)$$

- *Kernel de base radial*

$$K(x, x') = \exp(-\gamma \|x - x'\|^2) \quad (2.39)$$

- *Kernel de redes neuronales*

$$K(x, x') = \tanh(\kappa_1 \langle x, x' \rangle + \kappa_2) \quad (2.40)$$

Con $\kappa_1 > 0$ y $\kappa_2 \in \mathbb{R}$ Una vez establecido el kernel el problema se reduce a encontrar los $\hat{\alpha}_i$ que optimizan el problema dual:

$$\max \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{i'=1}^N \alpha_i \alpha_{i'} y_i y_{i'} K(x_i, x_{i'}) \quad (2.41)$$

Sujeto a

$$\sum_{i=1}^N \alpha_i y_i = 0 \quad 0 \leq \alpha_i \leq C \quad i = 1, \dots, N \quad (2.42)$$

Así, la función de decisión lineal vendrá expresada como solución del problema de optimización anterior, en términos del kernel K y el costo C . Es importante destacar que, como en las secciones anteriores, la solución al problema dual se puede representar únicamente mediante los vectores de soporte, por lo que este problema no depende de la dimensionalidad del espacio de características, sino solo de la cardinalidad del conjunto de vectores de soporte.

Podemos determinar el valor β_0 usando los vectores de soporte que se encuentran en el margen, es decir $0 < \alpha_i < C$, resolviendo la ecuación

$$1 = y_i f(x_i) = y_i \left(\sum_{j=1}^N \alpha_j y_j K(x_i, x'_j) + \beta_0 \right) \quad (2.43)$$

Un valor grande de C penaliza fuertemente los errores de clasificación, limitando la cantidad de puntos mal clasificados o que estén dentro del margen. Esto puede llevar a un sobreajuste en el espacio de características. Por otro lado, un valor pequeño de C permite más errores de clasificación, lo que genera un valor más pequeño de β y da lugar a un modelo con una frontera de decisión más suave.

Para ejemplificar esta sección sobre kernels consideremos el siguiente ejemplo,

(x_1, x_2)	y
$(+1, +1)$	$+1$
$(-1, +1)$	-1
$(-1, -1)$	$+1$
$(+1, -1)$	-1

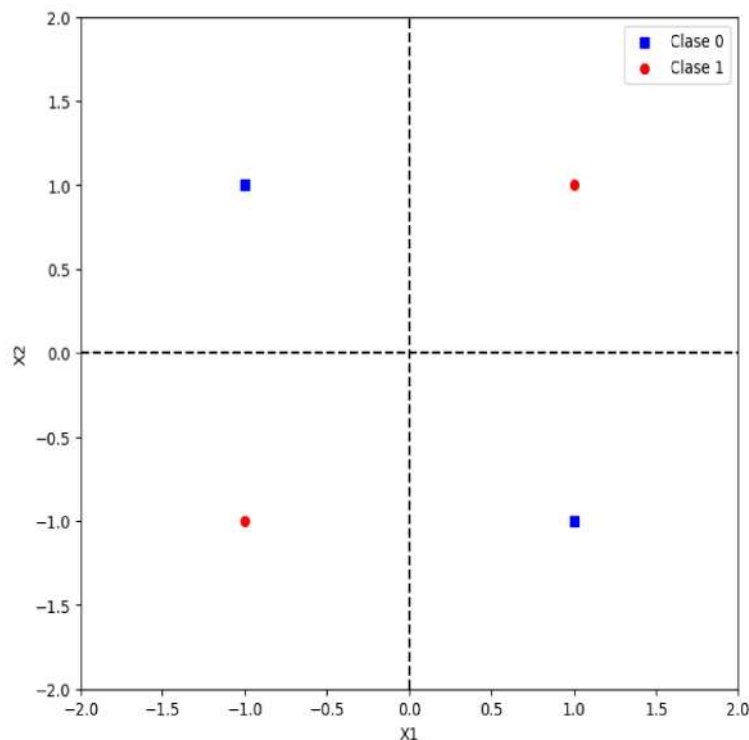


Figura 2.5: Los datos no son linealmente separables pues no existe una recta que separe los datos de manera eficiente, pues cualquiera que se intente trazar, dejara a al menos un elemento fuera de su clasificación correspondiente.

Es fácil ver en la figura 2.5 que los datos no son linealmente separables, por lo que es necesario aplicar nuestro método de kernels para ampliar el espacio de entradas

a un espacio de características en el que este problema sea linealmente separable. Se propone usar un kernel polinómico de grado dos, es decir,

$$K(x, x') = \langle h(x), h(x') \rangle = [1 + \langle x, x' \rangle]^2 \quad (2.44)$$

Veamos pues explícitamente quiénes son las funciones base h , mencionadas en la definición del kernel en la ecuación (1.9), entonces de (2.44) tenemos

$$\begin{aligned} \langle h(x), h(x') \rangle &= [1 + \langle x, x' \rangle]^2 = [1 + \langle (x_1, x_2), (x'_1, x'_2) \rangle]^2 \\ &= [1 + x_1 x'_1 + x_2 x'_2]^2 \\ &= 1 + 2x_1 x'_1 + 2x_2 x'_2 + 2x_1 x'_1 x_2 x'_2 + x_1^2 (x'_1)^2 + x_2^2 (x'_2)^2 \\ &= \left\langle \begin{bmatrix} 1, \sqrt{2}x_1, \sqrt{2}x_2, \sqrt{2}x_1 x_2, x_1^2, x_2^2 \end{bmatrix}, \begin{bmatrix} 1, \sqrt{2}x'_1, \sqrt{2}x'_2, \sqrt{2}x'_1 x'_2, (x'_1)^2, (x'_2)^2 \end{bmatrix} \right\rangle \end{aligned}$$

Por lo tanto las funciones bases son

$$\begin{aligned} h_1(x_1, x_2) &= 1 \\ h_2(x_1, x_2) &= \sqrt{2}x_1 \\ h_3(x_1, x_2) &= \sqrt{2}x_2 \\ h_4(x_1, x_2) &= \sqrt{2}x_1 x_2 \\ h_5(x_1, x_2) &= x_1^2 \\ h_6(x_1, x_2) &= x_2^2 \end{aligned}$$

Así, nuestro problema en dos dimensiones lo estamos mapeando a un espacio de seis dimensiones. El problema se reduce a encontrar los α_i de las ecuaciones (2.41) y (2.42). La solución de este problema de optimización viene dada por $\alpha_i = 0.125$ para $i = 1, 2, 3, 4$, es decir, que todos los ejemplos del conjunto de entrenamiento corresponden a vectores de soporte. Por lo tanto, la función de decisión vendrá dada por

$$\begin{aligned}
f(x) &= \sum_{i=1}^4 \hat{\alpha}_i y_i K(x, x_i) + \hat{\beta}_0 = 0.125 \sum_{i=1}^4 y_i K(x, x_i) + \hat{\beta}_0 \\
&= 0.125 \left[4\sqrt{2}h_4(x) \right] = \frac{1}{\sqrt{2}}h_4(x)
\end{aligned}$$

Entonces, la función de decisión queda determinada en términos de solo una dimensión, h_4 . Por lo tanto, basta con una dimensión del espacio transformado para separar los cuatro datos. Para obtener la ecuación del hiperplano, basta con igualar a cero la ecuación anterior

$$\frac{1}{\sqrt{2}}h_4(x) = 0,$$

e igualar a 1 y -1 para obtener los márgenes. Así, en el espacio original, la función de decisión no lineal es $x_1x_2 = 0$ y los márgenes los delimitan las hipérbolas $x_1x_2 = 1$ y $x_1x_2 = -1$, que se muestran en la figura 2.6.

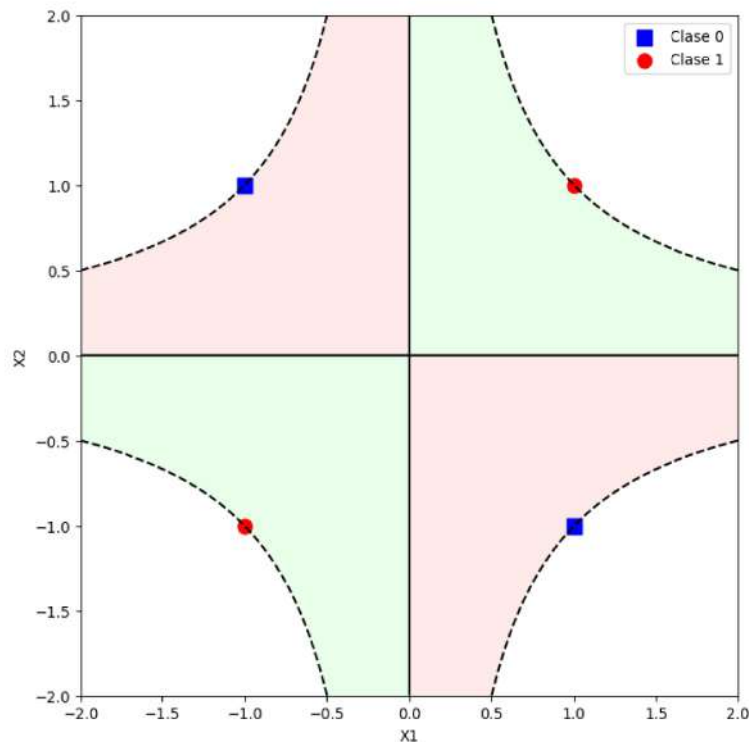


Figura 2.6: El problema a pesar de ser mapeado en un espacio de 6 dimensiones, solo ocupa una para separar los datos h_4 , y son las hipérbolas $x_1x_2 = 1$ y $x_1x_2 = -1$, las que determinan nuestro hiperplano separador

Capítulo 3

Penalización y Nucleos Reproducibles

3.1. SVM como metodo de penalización

En algunas ocasiones, nos es conveniente ver el SVM como un modelo de penalización, donde el objetivo es minimizar el error en la clasificación de los datos de entrenamiento e intentar controlar la complejidad del modelo. Este enfoque nos permite penalizar al modelo cuando realiza una mala clasificación y también nos ayuda a evitar el sobreajuste.

En este sentido, podemos ver el problema como

$$\min_{\beta, \beta_0} \sum_{i=1}^N [1 - y_i f(x_i)]_+ + \frac{\lambda}{2} \|\beta\|^2 \quad (3.1)$$

Pues recordemos que el problema original viene dado por

$$\min_{\beta, \beta_0} \frac{1}{2} \|\beta\|^2 + C \sum_{i=1}^N \xi_i \quad \text{sujeto a} \quad \xi_i \geq 0 \quad \text{y} \quad y_i(x_i^T \beta + \beta_0) \geq 1 - \xi_i \quad \forall i \quad (3.2)$$

Entonces de la restricción

$$\begin{aligned} y_i f(x_i) = y_i(x_i^T \beta + \beta_0) &\geq 1 - \xi_i \\ \Rightarrow \xi_i &\geq 1 - y_i(x_i^T \beta + \beta_0) \end{aligned}$$

Podemos definir $\xi_i = \max(0, 1 - y_i f(\mathbf{x}_i)) = [1 - y_i f(x_i)]_+$, esta usualmente es llamada la función hinge loss como se menciona en la sección 1.7 (es natural definir la función de esa forma, pues ξ_i nos indica el error; y para los puntos correctamente clasificados no estamos considerando un error, por lo que el valor en la función es 0. Contrario a esto, para aquellos puntos que sí están mal clasificados, la función toma un valor mayor que 0), por lo que, sustituyendo en (3.2), se obtiene

$$\min_{\beta, \beta_0} \frac{1}{2} \|\beta\|^2 + C \sum_{i=1}^N [1 - y_i f(x_i)]_+ \quad \text{sujeto a} \quad [1 - y_i f(x_i)]_+ \geq 0 \quad (3.3)$$

Ahora si consideramos $C = \frac{1}{\lambda}$

$$\begin{aligned} \min_{\beta, \beta_0} \quad & \frac{1}{2} \|\beta\|^2 + \frac{1}{\lambda} \sum_{i=1}^N [1 - y_i f(x_i)]_+ \\ \Rightarrow \min_{\beta, \beta_0} \quad & \frac{\lambda}{2} \|\beta\|^2 + \sum_{i=1}^N [1 - y_i f(x_i)]_+ \end{aligned} \quad (3.4)$$

Donde el término $[1 - y_i f(x_i)]_+$ es la función hinge loss (Pérdida de bisagra), y $[]_+$ indica la parte positiva; note que podemos eliminar la restricción, pues por la naturaleza de la función esta siempre sera positiva; más explícitamente esta función está definida por

$$\ell^{\text{hinge}}(f(x), y) = \begin{cases} 0 & \text{si } yf(x) \geq 1 \\ 1 - yf(x) & \text{en otro caso} \end{cases} \quad (3.5)$$

Ahora notemos que, para puntos fuera del margen, es decir, aquellos para los cuales se cumple que $y_i f(x_i) > 1$, no tienen contribución a la función. Para los vectores de soporte, es decir, para aquellos para los cuales $y_i f(x_i) = 1$, la función tampoco contribuye a la pérdida. Finalmente, para los puntos dentro del margen se cumple que $y_i f(x_i) < 1$, y por lo tanto su contribución a la función Hinge loss es $1 - y_i f(x_i)$.

Y la segunda parte es un término de penalización $\frac{\lambda}{2} \|\beta\|^2$ que castiga a los modelos más complejos. Aquí el parámetro λ sirve como controlador de la magnitud de la penalización aplicada a los coeficientes del modelo β . De cierta forma, determina el equilibrio entre aumentar el tamaño del margen y asegurarse de que x_i esté en el lado

correcto del margen. Aunque la ecuación (3.4) no es diferenciable, podemos encontrar su punto mínimo mediante el método de descenso del gradiente (Oscar Ramos, 2020).

En este contexto, el SVM combina una función de pérdida con una de penalización (Pérdida + Penalización). Esta combinación busca minimizar tanto el error de los datos de entrenamiento como la complejidad del modelo para dar la mejor generalización posible. La función Hinge loss es la más adecuada para la clasificación binaria, en comparación con otras funciones de pérdida más tradicionales, como lo son la desviación binomial o el error cuadrático. Esto se ha demostrado mediante la práctica (e.g. Hastie & Tibshirani).

En la ecuación anterior, se presenta al SVM como un problema de estimación de función regularizada (ver Capítulo 17). Las funciones de pérdida, como la desviación binomial, *SVM hinge loss* y *Square Hinge Loss*, son llamadas funciones de pérdida que maximizan el margen.

Al hacer que los coeficientes de la expansión lineal $f(x) = \beta_0 + h(x)^T \beta$ (excluyendo β_0) se reduzcan hacia cero, imponemos una penalización sobre la magnitud de los coeficientes de β , lo que nos ayuda a evitar un posible sobreajuste y a simplificar el modelo. Cuando el parámetro de regularización λ es muy grande, la penalización es mayor, lo que restringe la magnitud de los coeficientes β y lleva a un modelo más simple, posiblemente con un margen más pequeño. Mientras que si λ tiende a cero, la penalización se vuelve muy pequeña y el modelo se enfoca más en minimizar la función de pérdida sin preocuparse por la complejidad del modelo. Cuando los datos son separables y $\lambda \rightarrow 0$, la penalización en los coeficientes se reduce, lo que genera un hiperplano que maximiza el margen.

3.2. SVM en terminos de RKHS

Ahora nos centraremos en describir el SVM en términos de estimación de funciones en espacios de Hilbert con núcleos reproductores (RKHS, por sus siglas en inglés). La ventaja es que ofrece una forma estructurada de trabajar con funciones en un espacio posiblemente infinito, gracias a la propiedad de reproducción del núcleo. Esta parte

sustenta formalmente la sección 3 del capítulo 2, por lo que puede ser omitido en un principio.

Supongamos una base h , que surge de la expansión en eigenvalores (Valores propios) de un kernel definido positivo K

$$K(x, x') = \sum_{m=1}^{\infty} \phi_m(x) \phi_m(x') \delta_m \quad (3.6)$$

y $h_m(x) = \sqrt{\delta_m} \phi_m(x)$ Si hacemos $\theta_m = \sqrt{\delta_m} \beta_m$ podemos escribir la función de penalización como

$$\min_{\beta, \beta_0} \sum_{i=1}^N [1 - y_i f(x_i)]_+ + \frac{\lambda}{2} \|\beta\|^2 = \min_{\beta_0, \theta} \sum_{i=1}^N \left[1 - y_i \left(\beta_0 + \sum_{m=1}^{\infty} \theta_m \phi_m(x_i) \right) \right]_+ + \frac{\lambda}{2} \sum_{m=1}^{\infty} \frac{\theta_m^2}{\delta_m} \quad (3.7)$$

Qué es similar a la ecuación (1.17) descrita en la sección 7 del capítulo 1, ademas los espacios de Hilbert con núcleo reproductor garantizan una solución de dimensión finita en la forma:

$$f(x) = \beta_0 + \sum_{i=1}^N \alpha_i K(x, x_i) \quad (3.8)$$

En particular hay una versión equivalente al criterio de optimización (2.30) dado por

$$\min_{\beta_0, \alpha} \sum_{i=1}^N (1 - y_i f(x_i))_+ + \frac{\lambda}{2} \alpha^T \mathbf{K} \alpha \quad (3.9)$$

Donde $\mathbf{K}$ es una matriz de $N \times N$ donde se evalúan todos los pares de características de entrada en el kernel.

De la ecuación (3.8) podemos ver que el hiperplano separador está dado por las evaluaciones de las observaciones el nucleo K y este a su vez esta determinado por los productos internos de las funciones base h_i .

Más generalmente podemos expresar el problema como

$$\min_{f \in \mathcal{H}} \sum_{i=1}^N [1 - y_i f(x_i)]_+ + \lambda J(f) \quad (3.10)$$

donde $\mathcal{H}$ es el espacio estructurado de funciones, y $J(f)$ es un regularizador adecuado en ese espacio. Por ejemplo, podríamos usar el espacio de funciones aditivas, por lo que el kernel vendría dado por $K(x, x') = \sum_{j=1}^p K_j(x_j, x'_j)$, y cada K_j es el kernel apropiado para el spline suavizado univariado en x_j (para más detalles, consultar Wahba, 1990).

Antes de terminar esta sección daremos algunas observaciones importantes que vale la pena destacar, la primera es que cualquiera de los núcleos descritos en (2.38)–(2.40), pueden usarse con cualquier función de pérdida convexa, lo que nos llevará a una representación de la solución de dimensión finita de la forma (3.8).

Para los SVM, puede que una fracción considerable de los N valores de α_i puedan ser ceros y no influyan en la decisión final del clasificador. Esto es una consecuencia de la naturaleza lineal por partes de la función hinge loss.

Generalmente, reducir λ reducirá la superposición, lo que puede hacer que el modelo sea más flexible, pero un exceso de flexibilidad puede hacer que el modelo se ajuste demasiado a los datos de entrenamiento.

Por último, destacamos que la elección del kernel depende en gran medida de la distribución de los datos. En términos generales, cada kernel ofrece ventajas y desventajas según la naturaleza del conjunto de datos y el tipo de relaciones que se desean capturar.

El kernel lineal es adecuado cuando los datos presentan una separación lineal evidente o cuando se dispone de un número muy elevado de características. Su principal ventaja es el bajo costo computacional, ya que no transforma el espacio de entrada, lo que ayuda a evitar el sobreajuste. Además, el modelo resultante suele ser más interpretable, facilitando la comprensión de la relación entre las variables.

Por otro lado, el kernel polinomial es útil cuando se sospecha que la relación entre los datos puede modelarse mediante polinomios de grado d . Este enfoque permite capturar relaciones no lineales entre las variables gracias al grado del polinomio: si d es bajo, se obtiene una flexibilidad moderada, mientras que valores altos permiten modelar relaciones más complejas. Sin embargo, un polinomio de grado elevado puede llevar al sobreajuste. Además, este kernel presenta un mayor costo computacional, ya que requiere calcular potencias y realizar transformaciones a espacios de dimensión superior,

lo que puede incrementar significativamente el tiempo de entrenamiento, especialmente en conjuntos de datos grandes.

El kernel RBF (Radial Basis Function) es particularmente útil cuando los datos tienen distribuciones complejas y no pueden separarse de manera lineal o polinómica, como se ilustra en la Figura 2.4. Este kernel permite aproximar cualquier función de decisión con los parámetros adecuados, ya que transforma los datos a un espacio de dimensión infinita. No obstante, esta capacidad de capturar relaciones altamente complejas también incrementa el riesgo de sobreajuste si el parámetro γ no se elige correctamente. Si γ es demasiado alto, el modelo puede ajustarse en exceso a los datos de entrenamiento, mientras que si es demasiado bajo, el modelo puede no capturar suficiente información, llevando al subajuste.

El kernel de red neuronal es adecuado cuando los datos presentan una combinación de relaciones lineales y no lineales, con transiciones suaves entre clases, similares a la función de activación sigmoideal. Es decir, cuando la relación entre las variables se asemeja a la activación de una neurona, donde la frontera de decisión no cambia abruptamente, sino de manera progresiva. En cuanto al costo computacional, tiene la ventaja de no inducir una transformación en un espacio de dimensiones extremadamente alto. Sin embargo, al igual que otros kernels no lineales, requiere un ajuste preciso de los parámetros para evitar tanto el sobreajuste como el subajuste.

Schölkopf y Smola (2002), destacan que una tarea igual de importante que elegir el kernel, es determinar los parámetros óptimos de cada kernel para obtener una función de decisión adecuada. No existe una respuesta absoluta a cómo obtener estos parámetros, pero una estrategia común es emplear técnicas como la validación cruzada para ajustar los parámetros y minimizar tanto el subajuste como el sobreajuste,

Por ejemplo, en el conjunto de datos MNIST, los autores presentan diferentes clasificadores entrenados con tres tipos de kernels y distintos valores de parámetros. Para normalizar los píxeles de las imágenes, los datos se dividen entre 256 antes del entrenamiento. Los resultados obtenidos se resumen en la siguiente tabla.

Note que, con cualquier kernel y una selección adecuada de parámetros, se puede alcanzar un porcentaje de error del 4%. Esto resalta la importancia de un ajuste

Kernel Polinomial: $k(x, x') = \left(\frac{\langle x, x' \rangle}{256}\right)^d$							
d	1	2	3	4	5	6	7
Tasa de error %	8.9	4.7	4.0	4.2	4.5	4.5	4.7
Vectores de Soporte	282	237	274	321	374	422	491

Kernel RBF: $k(x, x') = \exp(-\gamma\ x - x'\ ^2/(256))$							
γ	0.25	0.5	0.85	1.25	2	5	10
Tasa de error %	5.3	5.0	4.9	4.3	4.4	4.4	4.5
Vectores de Soporte	266	240	233	253	251	366	722

adecuado de los parámetros, con el fin de mejorar el desempeño del modelo.

Kernel de red neuronal: $k(x, x') = \tanh(2\langle x, x' \rangle / 256 + \kappa_2)$							
$-\kappa_2$	0.8	0.9	1.0	1.1	1.2	1.3	1.4
Tasa de error %	6.3	4.8	4.8	4.1	4.3	4.4	4.8
Vectores de Soporte	206	242	254	267	278	289	296

Capítulo 4

Envenenamiento de SVM

4.1. Envenenamiento

Una vez establecido cómo funciona el modelo SVM, nos enfocaremos en todos los conceptos relacionados con el envenenamiento de las Máquinas de Soporte Vectorial. Como mencionamos en el capítulo 1, existen diferentes métodos para atacar el rendimiento de un modelo de machine learning, y es importante que un modelo sea robusto ante tales situaciones, pues podría haber involucrado información importante.

Supongamos que tenemos un conjunto de datos de entrenamiento $D_{tr} = \{(x_i, y_i)\}_{i=1}^n$. Denotemos por $Q = yy^T \odot \mathbf{K}$ a la matriz de etiquetas de K , donde $\mathbf{K}$ es la matriz de evaluación del kernel (en este contexto, $\odot$ indica el producto entrada a entrada de las matrices). Notemos que $Q_{ij} = y_i y_j K(x_i, x_j)$ es simétrica y definida positiva, pues K es un kernel definido positivo.

Dependiendo del valor de α_i , podemos clasificar los puntos de entrenamiento en tres conjuntos:

$$S = \{x_i \mid 0 < \alpha_i < C\} \quad \text{Vectores de soporte de margen}$$

$$E = \{x_i \mid \alpha_i = C\} \quad \text{Vectores de error de soporte}$$

$$R = \{x_i \mid \alpha_i = 0\} \quad \text{Puntos de reserva}$$

Usaremos las letras minúsculas s , e , r para indexar los puntos correspondientes de vectores o matrices. Por ejemplo, Q_{ss} denota la submatriz Q correspondiente a los vectores de soporte del margen.

Asumiendo el rol del atacante, nuestro objetivo es encontrar un punto (x_c, y_c) que, al ser añadido al conjunto de entrenamiento, disminuya la precisión del clasificador SVM. La elección de la etiqueta del punto de ataque y_c es arbitraria pero fija. Nos referimos a la clase de esta etiqueta como clase atacante y a la otra como clase atacada.

Procedemos primero extrayendo un conjunto de datos de validación $D_{val} = \{(x_i, y_i)\}_{i=1}^m$ nuestro objetivo es maximizar la pérdida hinge que el SVM comete sobre los puntos en el conjunto de validación, donde el SVM se entrena con $D_{tr} \cup \{(x_c, y_c)\}$.

$$\max_{x_c} L(x_c) = \sum_{k=1}^m (1 - y_k f_{x_c}(x_k))_+ = \sum_{k=1}^m (-g_k)_+ \quad (4.1)$$

Considerando todos los términos en las condiciones del margen g_k que son afectados por x_c , es decir, aquellos que cambian conforme x_c varia. Entonces

Note que la función $L(x_c)$ es una función objetivo no convexa, pero puede optimizarse mediante el método de ascenso del gradiente.

Este método consiste en escoger inicialmente un punto de ataque $x_c^{(0)}$ y, después, actualizar este punto de ataque mediante $x_c^{(p)} = x_c^{(p-1)} + tu$, donde p es el número de iteración, u es un vector de norma uno que representa la dirección del ataque y t es el tamaño del paso que se da en cada iteración. La dirección del ataque de u debe alinearse con el gradiente de la función L con respecto a u , y esta debe calcularse en cada iteración.

Aunque la función hinge loss no es diferenciable en todos sus puntos debido a que es una función definida por partes, podemos evitar este problema centrándonos únicamente en los puntos k que tienen una contribución no nula a L . En otras palabras, consideramos sólo los puntos (x_k, y_k) que están mal clasificados, aquellos para los cuales $-g_k > 0$.

Las contribuciones que hacen estos puntos al gradiente pueden calcularse diferenciando (??) con respecto a u , lo que nos ayuda a ver cómo cambia L en función de la dirección de u .

$$\begin{aligned}
\frac{\partial g_k}{\partial u} &= \sum_{j \in S} \frac{\partial}{\partial u} (Q_{kj} \alpha_j) + y_k \frac{\partial b}{\partial u} \\
&= \sum_j Q_{kj} \frac{\partial \alpha_j}{\partial u} + \sum_j \alpha_j \frac{\partial Q_{kj}}{\partial u} + y_k \frac{\partial b}{\partial u} \\
&= Q_{k1} \left[\frac{\partial \alpha_1}{\partial u_1}, \dots, \frac{\partial \alpha_1}{\partial u_d} \right] + Q_{k2} \left[\frac{\partial \alpha_2}{\partial u_1}, \dots, \frac{\partial \alpha_2}{\partial u_d} \right] + \dots + Q_{ks} \left[\frac{\partial \alpha_s}{\partial u_1}, \dots, \frac{\partial \alpha_s}{\partial u_d} \right] + \\
&\quad \left[\frac{\partial Q_{k1}}{\partial u_1}, \dots, \frac{\partial Q_{k1}}{\partial u_d} \right] \alpha_1 + \left[\frac{\partial Q_{k2}}{\partial u_1}, \dots, \frac{\partial Q_{k2}}{\partial u_d} \right] \alpha_2 + \dots + \left[\frac{\partial Q_{ks}}{\partial u_1}, \dots, \frac{\partial Q_{ks}}{\partial u_d} \right] \alpha_s + y_k \frac{\partial b}{\partial u} \\
&= \left[Q_{k1} \frac{\partial \alpha_1}{\partial u_1} + \dots + Q_{ks} \frac{\partial \alpha_s}{\partial u_1}, Q_{k1} \frac{\partial \alpha_1}{\partial u_2} + \dots + Q_{ks} \frac{\partial \alpha_s}{\partial u_2}, \dots, Q_{k1} \frac{\partial \alpha_1}{\partial u_d} + \dots + Q_{ks} \frac{\partial \alpha_s}{\partial u_d} \right] + \\
&\quad \left[\frac{\partial Q_{k1}}{\partial u_1} \alpha_1 + \dots + \frac{\partial Q_{ks}}{\partial u_1} \alpha_s, Q_{k1} \frac{\partial Q_{k1}}{\partial u_2} \alpha_1 + \dots + \frac{\partial Q_{ks}}{\partial u_2} \alpha_s, \dots, \frac{\partial Q_{k1}}{\partial u_d} \alpha_1 + \dots + \frac{\partial Q_{ks}}{\partial u_d} \alpha_s \right] + y_k \frac{\partial b}{\partial u} \\
&= Q_{ks} \frac{\partial \alpha}{\partial u} + \frac{\partial Q_{kc}}{\partial u} \alpha_c + y_k \frac{\partial b}{\partial u}
\end{aligned}$$

Por lo tanto

$$\frac{\partial g_k}{\partial u} = Q_{ks} \frac{\partial \alpha}{\partial u} + \frac{\partial Q_{kc}}{\partial u} \alpha_c + y_k \frac{\partial b}{\partial u} \quad (4.2)$$

Donde

$$\frac{\partial \alpha}{\partial u} = \begin{bmatrix} \frac{\partial \alpha_1}{\partial u_1} & \dots & \frac{\partial \alpha_1}{\partial u_d} \\ \vdots & \ddots & \vdots \\ \frac{\partial \alpha_s}{\partial u_1} & \dots & \frac{\partial \alpha_s}{\partial u_d} \end{bmatrix}, \quad \frac{\partial Q_{kc}}{\partial u} = \left[\frac{\partial Q_{kc}}{\partial u_1}, \dots, \frac{\partial Q_{kc}}{\partial u_d} \right], \quad \frac{\partial b}{\partial u} = \left[\frac{\partial b}{\partial u_1}, \dots, \frac{\partial b}{\partial u_d} \right].$$

Este es el gradiente que nos dará la dirección de ataque, sin embargo esta expresión puede refinarse aún más usando el hecho de que el paso tomado en la dirección u debe mantener la solución óptima del SVM. Esto puede formularse como una condición de actualización adiabática (Cauwenberghs & Poggio, 2001).

Para refinar más las expresiones para el gradiente es importante destacar que, para el punto i -ésimo en el conjunto de entrenamiento, las condiciones de KKT para la

solución óptima pueden expresarse como

$$g_i = \sum_{j \in D_{tr}} Q_{ij} \alpha_j + y_i b - 1 = \begin{cases} > 0 & i \in R \\ = 0 & i \in S \\ < 0 & i \in E \end{cases} \quad (4.3)$$

$$h = \sum_{j \in D_{tr}} y_j \alpha_j = 0 \quad (4.4)$$

Esto implica que un cambio infinitesimal en las ecuaciones anteriores en el punto de ataque x_c causa un cambio suave en la solución óptima del SVM, manteniendo la composición de los conjuntos R, S y E constantes; mientras el cambio sea pequeño este equilibrio nos permite predecir cómo se ajustará la solución del SVM en respuesta a cambios en x_c . Al diferenciar los terminos de las ecuaciones anteriores dependientes de x_c con respecto a cada componente de la dirección de ataque ($1 \leq l \leq d$) para todos los puntos de soporte $i \in S$

$$\begin{aligned} \frac{\partial g_i}{\partial u_l} &= Q_{ss} \frac{\partial \alpha}{\partial u_l} + \frac{\partial Q_{sc}}{\partial u_l} \alpha_c + y_s \frac{\partial b}{\partial u_l} = 0 \\ \frac{\partial h}{\partial u_l} &= y_s^\top \frac{\partial \alpha}{\partial u_l} = 0 \end{aligned}$$

las cuales podemos reescribir como

$$\begin{aligned} y_s^\top \frac{\partial \alpha}{\partial u_l} &= 0 \\ Q_{ss} \frac{\partial \alpha}{\partial u_l} + y_s \frac{\partial b}{\partial u_l} &= - \frac{\partial Q_{sc}}{\partial u_l} \alpha_c \end{aligned}$$

En notación matricial lo podemos ver como

$$\begin{bmatrix} 0 & y_s^\top \\ y_s & Q_{ss} \end{bmatrix} \begin{bmatrix} \frac{\partial b}{\partial u_l} \\ \frac{\partial \alpha}{\partial u_l} \end{bmatrix} = - \begin{bmatrix} 0 \\ \frac{\partial Q_{sc}}{\partial u_l} \end{bmatrix} \alpha_c$$

o bien

$$\begin{bmatrix} \frac{\partial b}{\partial u_l} \\ \frac{\partial \alpha}{\partial u_l} \end{bmatrix} = - \begin{bmatrix} 0 & y_s^\top \\ y_s & Q_{ss} \end{bmatrix}^{-1} \begin{bmatrix} 0 \\ \frac{\partial Q_{sc}}{\partial u_l} \end{bmatrix} \alpha_c \quad (4.5)$$

Notemos que podemos calcular la inversa de la primera matriz, en la parte derecha de la igualdad anterior, mediante la formula de Sherman-Morrison-Woodburg.

$$\begin{bmatrix} 0 & y_s^\top \\ y_s & Q_{ss} \end{bmatrix}^{-1} = \begin{bmatrix} -(y_s^\top Q_{ss}^{-1} y_s)^{-1} & (y_s^\top Q_{ss}^{-1} y_s)^{-1} Q_{ss}^{-1} y_s^\top \\ (y_s^\top Q_{ss}^{-1} y_s)^{-1} Q_{ss}^{-1} y_s & Q_{ss}^{-1} - Q_{ss}^{-1} y_s (Q_{ss}^{-1} y_s)^\top \end{bmatrix}$$

Si hacemos $\mathbf{v} = Q_{ss}^{-1} y_s$ y $\zeta = y_s^\top Q_{ss}^{-1} y_s$ obtenemos:

$$\begin{bmatrix} 0 & y_s^\top \\ y_s & Q_{ss} \end{bmatrix}^{-1} = \frac{1}{\zeta} \begin{bmatrix} -1 & \mathbf{v}^\top \\ \mathbf{v} & \zeta Q_{ss}^{-1} - \mathbf{v} \mathbf{v}^\top \end{bmatrix}$$

Sustituyendo esta expresión en la ecuación (4.5) tenemos:

$$\begin{bmatrix} \frac{\partial b}{\partial u_l} \\ \frac{\partial \alpha}{\partial u_l} \end{bmatrix} = -\frac{1}{\zeta} \begin{bmatrix} -1 & \mathbf{v}^\top \\ \mathbf{v} & \zeta Q_{ss}^{-1} - \mathbf{v} \mathbf{v}^\top \end{bmatrix} \begin{bmatrix} 0 \\ \frac{\partial Q_{sc}}{\partial u_l} \end{bmatrix} \alpha_c$$

de donde

$$\begin{aligned} \frac{\partial b}{\partial u} &= -\frac{1}{\zeta} \alpha_c \mathbf{v}^\top \cdot \frac{\partial Q_{sc}}{\partial u} \\ \frac{\partial \alpha}{\partial u} &= -\frac{1}{\zeta} \alpha_c (\zeta Q_{ss}^{-1} - \mathbf{v} \mathbf{v}^\top) \cdot \frac{\partial Q_{sc}}{\partial u} \end{aligned}$$

Una vez calculadas estas expresiones podemos sustituir las en la ecuación (4.2)

$$\begin{aligned} \frac{\partial g_k}{\partial u} &= Q_{ks} \frac{\partial \alpha}{\partial u} + \frac{\partial Q_{kc}}{\partial u} \alpha_c + y_k \frac{\partial b}{\partial u} \\ &= -Q_{ks} \frac{1}{\zeta} \alpha_c (\zeta Q_{ss}^{-1} - \mathbf{v} \mathbf{v}^\top) \cdot \frac{\partial Q_{sc}}{\partial u} - y_k \frac{1}{\zeta} \alpha_c \mathbf{v}^\top \cdot \frac{\partial Q_{sc}}{\partial u} + \frac{\partial Q_{kc}}{\partial u} \alpha_c \end{aligned}$$

Tomando $M_k = -\frac{1}{\zeta} (Q_{ks} (\zeta Q_{ss}^{-1} - \mathbf{v}\mathbf{v}^\top) + y_k \mathbf{v}^\top)$ obtenemos

$$\frac{\partial g_k}{\partial u} = M_k \frac{\partial Q_{sc}}{\partial u} \alpha_c + \frac{\partial Q_{kc}}{\partial u} \alpha_c = \left(M_k \frac{\partial Q_{sc}}{\partial u} + \frac{\partial Q_{kc}}{\partial u} \right) \alpha_c$$

Sustituyendo esta expresión en (4.1) obtenemos el gradiente deseado que se usa para optimizar nuestro ataque

$$\frac{\partial L}{\partial u} = \sum_{k=1}^m \frac{\partial g_k}{\partial u} = \sum_{k=1}^m \left\{ M_k \frac{\partial Q_{sc}}{\partial u} + \frac{\partial Q_{kc}}{\partial u} \right\} \alpha_c \quad (4.6)$$

De la ecuación (4.6) podemos notar que el gradiente de la función objetivo en cada iteración k depende del punto de ataque $x_c^{(p)}$ sólo a través de los gradientes de la matriz Q , y dado que esta está dada en términos del kernel elegido, entonces dependerá del kernel.

Las expresiones para los gradientes de los tres kernels más comunes son

■ **Kernel lineal:**

$$\frac{\partial K_{ic}}{\partial u} = \frac{\partial (x_i \cdot x_c^{(p)})}{\partial u} = t x_i$$

■ **Kernel polinomial:**

$$\frac{\partial K_{ic}}{\partial u} = \frac{\partial (x_i \cdot x_c^{(p)} + R)^d}{\partial u} = d(x_i \cdot x_c^{(p)} + R)^{d-1} t x_i$$

■ **Kernel RBF:**

$$\frac{\partial K_{ic}}{\partial u} = \frac{\partial e^{-\frac{\gamma}{2} \|x_i - x_c^{(p)}\|^2}}{\partial u} = K(x_i, x_c^{(p)}) \gamma t (x_i - x_c^{(p)})$$

La dependencia de $x_c^{(p)}$ y, por lo tanto, de u en los gradientes de los kernels no lineales puede evitarse sustituyendo $x_c^{(p)}$ con su valor en la iteración anterior $x_c^{(p-1)}$, siempre que el tamaño de paso t sea lo suficientemente pequeño. Esto nos asegura que, en cada paso de ajuste, asumimos que el cambio es mínimo, y así, podemos hacer una aproximación

en los cálculos sin una pérdida significativa de precisión.

Como mencionamos anteriormente, el algoritmo depende del proceso iterativo del descenso del gradiente y se ajusta en cada paso siguiendo la dirección del gradiente. Este proceso comienza eligiendo un punto inicial $x_c^{(0)}$ de la clase objetivo a la que se quiere atacar (clase atacada). Este punto es simplemente una copia de un punto ya existente en el conjunto de entrenamiento, pero se le invierte la etiqueta de clase. Es importante que el punto esté lo suficientemente dentro de la clase atacada y que no esté muy cerca de la frontera, ya que un punto demasiado cercano a la frontera puede hacer que se convierta en uno de reserva, lo que detendría el progreso.

Posteriormente, con el punto de ataque inicializado, se entrena una SVM usando el conjunto de datos de entrenamiento D_{tr} sin el punto de ataque $x_c^{(0)}$, obteniendo así los parámetros α_i y el sesgo b_0 .

En cada iteración, el modelo se vuelve a entrenar usando un proceso incremental que incluye el punto de ataque actual (esta técnica está descrita por Poggio y Cauwenberghs, describimos a grandes rasgos sobre el aprendizaje incremental en el apéndice A). Esta técnica nos permite actualizar los parámetros del modelo sin tener que rehacer el entrenamiento desde cero.

Se calcula el gradiente de la función de pérdida para el conjunto de validación con respecto a la dirección de ajuste del punto de ataque.

Posteriormente, el gradiente es normalizado con una longitud unitaria y el punto de ataque se ajusta sumando un pequeño paso t en la dirección del gradiente. Esto se seguirá repitiendo hasta que la mejora en la pérdida del conjunto de validación entre una iteración y la consecutiva sea menor que un umbral ε .

Hay que recalcar que el cálculo del gradiente depende de que la estructura de los conjuntos S , E y R no cambien con cada pequeño ajuste del punto de ataque $x_c^{(p)}$, ya que al cambiar un punto de un conjunto al otro haría que tuviésemos que recalcular el gradiente, lo que podría resultar en un costo computacional muy alto.

También es necesario considerar un pequeño tamaño de paso t en la dirección de ajuste u , pues un paso demasiado grande podría hacer que algunos de los puntos también cambien de conjunto. En general, es difícil encontrar el valor máximo de t que

preserve la estructura de los conjuntos S , E y R .

Como mencionamos el algoritmo termina cuando el error de validación entre iteraciones es menor que un ε . Sin embargo, en ciertos kernels como el lineal, la superficie de error de validación puede crecer indefinidamente a medida que el punto de ataque se aleja de la región original de los datos, por lo que se le impone una restricción adicional al tamaño del ataque si el vector x_c se aleja demasiado de los puntos de entrenamiento, garantizando la convergencia. En el caso del Kernel RBF el valor de la evaluación en el kernel decae exponencialmente, esto forma una cota para los puntos lejanos en la función de decisión lo que facilita la búsqueda de máximos locales. Para el caso polinómico, la función puede crecer o decrecer según el grado y el parámetro usado, por lo que la superficie de error podría ser menos propensa a crecer sin límite en comparación con el kernel lineal, sin embargo también podría ocurrir lo contrario, en tal caso también se le impone una restricción. Esto nos asegura poder encontrar un máximo local y evitar soluciones que no tengan sentido práctico.

Finalmente, resumimos el algoritmo en el siguiente diagrama:

Algorithm 1 Poisoning attack against SVM

Require: Datos de entrenamiento $\mathcal{D}_{\text{tr}}$; Datos de validación $\mathcal{D}_{\text{val}}$; y_c la etiqueta de clase del punto de ataque; punto de ataque inicial $x_c^{(0)}$; y el tamaño del paso t .

Ensure: x_c , el punto final de ataque.

1: $\{\alpha_i, b\} \leftarrow$ entrenar el SVM con $\mathcal{D}_{\text{tr}}$.

2: $k \leftarrow 0$.

3: **repeat**

4: Re-calcular la solución del SVM con el conjunto $\mathcal{D}_{\text{tr}} \cup \{(x_c^{(p)}, y_c)\}$ usando SVM incremental (e.g., *Cauwenberghs & Poggio, 2001*). Para este paso ocupamos $\{\alpha_i, b\}$.

5: Calcular $\frac{\partial L}{\partial u}$ con $\mathcal{D}_{\text{val}}$ de acuerdo a la ecuación (4.6).

6: Establecer u un vector unitario alineado con $\frac{\partial L}{\partial u}$.

7: $k \leftarrow k + 1$ y $x_c^{(p)} \leftarrow x_c^{(p-1)} + tu$.

8: **until** $L(x_c^{(p)}) - L(x_c^{(p-1)}) < \varepsilon$.

9: **return** $x_c = x_c^{(p)}$.

Capítulo 5

Aplicaciones practicas

En la actualidad, debido a que el SVM es uno de los métodos más populares y eficientes en clasificación, se han creado múltiples paquetes informáticos que facilitan su aplicación a problemas existentes. Estas están diseñadas para atacar tanto los problemas de clasificación binaria como los multiclase, e incluso también existen librerías para solucionar problemas de regresión con SVM. Se ha popularizado el uso de Python y R para la solución de los problemas de Machine Learning; a continuación, presentamos las bibliotecas enfocadas en resolver este problema en cada software.

La biblioteca más popular para Machine Learning en Python es Scikit-Learn, que nos ofrece una implementación optimizada de SVM. Sin embargo, también existen otras librerías, como CVXPY, que está enfocada más en resolver el problema de optimización, y ThunderSVM, diseñada especialmente para acelerar el proceso de entrenamiento mediante GPU, útil en grandes cantidades de datos. Cada una tiene sus respectivas ventajas y desventajas.

Por otro lado, en R tenemos librerías como e1071, que nos posibilita personalizar kernels. kernlab también nos proporciona esta posibilidad, pero añade variantes como SVM multiclase y regresión. Sin embargo, si buscamos herramientas que también incluyen la búsqueda de hiperparámetros mediante validación cruzada, podemos optar por caret o mlr3. Pero para entrenar modelos con gran cantidad de datos, podemos optar por liquidSVM, que acelera la velocidad del entrenamiento.

Para las aplicaciones prácticas presentadas en este trabajo, estaremos usando el

software de Python.

En la presente sección, realizaremos un análisis de dos métodos de envenenamiento al dataset *digits* y otro análisis al conjunto de *Wine*, ambos disponibles en Scikit-Learn. Las librerías y los módulos necesarios son similares en ambos casos.

Primero, cargamos las librerías necesarias: NumPy nos ayuda a manejar arreglos multidimensionales, Matplotlib se utiliza para generar gráficos y scikit-learn para importar el modelo SVM.

```
1  import seaborn as sns
2  import pandas as pd
3  import numpy as np
4  import matplotlib
5  import matplotlib.pyplot as plt
6  from sklearn.model_selection import train_test_split
7  import importlib
8  from models.model import ScikitlearnClassifierSVC
9  from poisoning import PoisoningAttackSVM
10 from sklearn.svm import SVC
11 from sklearn.metrics import confusion_matrix,
12     ConfusionMatrixDisplay, accuracy_score
13 from sklearn import datasets, metrics
```

5.1. Aplicaciones: Digits

Para el primer caso, primero cargamos los datos *digits* desde Sklearn con el comando

```
1  digits = datasets.load_digits()
2  #digits
```

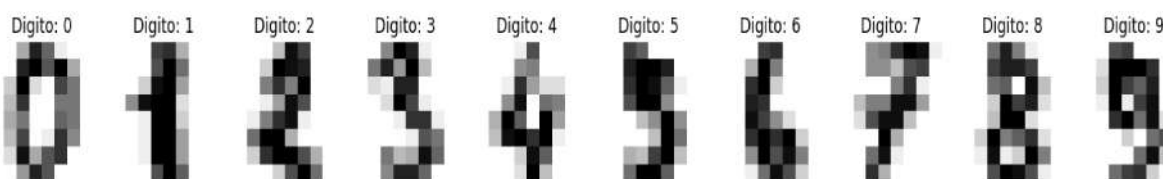
digits es un diccionario, pero nos enfocaremos en describir solo la información que nos será útil. Este dataset contiene 1797 imágenes, cada una con 64 filas que contienen la información de cada imagen correspondiente a un valor del color del píxel.

También tenemos una columna *target* que contiene las etiquetas de las 1797 imágenes (un número entre 0 y 9).

Este dataset también contiene la información en formato 8×8 , cada valor corresponde al color de los píxeles de las 1797 imágenes. Esta información es la misma que la de data, solo que en este caso son matrices de 8×8 , en vez de vectores de 64 entradas.

Mediante el siguiente código mostramos algunos dígitos.

```
1  _, axes = plt.subplots(nrows=1, ncols=10, figsize=(18, 4))
2  for ax, images, label in zip(axes, digits.images, digits.target):
3      ax.set_axis_off()
4      ax.imshow(images, cmap=plt.cm.gray_r, interpolation='nearest')
5      ax.set_title('Digito: ' + str(label))
```



Haremos un análisis de clasificación binaria para los dígitos 1 y 7, consideramos estos dígitos ya que la forma en que se escribe es similar, lo que nos ayudará en el ataque de envenenamiento.

Primero vamos a filtrar los números que vamos a ocupar para el análisis (1 y 7 en nuestro caso).

```
1  num = (digits.target == 1) | (digits.target == 7)
2  X = digits.data[num]
3  y = digits.target[num]
4  print(X.shape)
5  print(y.shape)
```

```
⇒ (361, 64)
   (361,)
```

Esto nos genera un arreglo de 361 filas por 64 columnas en el que contiene la información de los píxeles de todos los números 1's y 7's y un vector de 361 filas con sus respectivas etiquetas.

Ahora vamos a dividir a los conjuntos en entrenamiento, prueba y validación; para ello tomamos 70 datos para entrenamiento, 70 para prueba y 241 para validación,

```

1 X_train=X[:70]
2 y_train=y[:70]
3 X_test=X[70:140]
4 y_test=y[70:140]
5 X_val=X[120:]
6 y_val=y[120:]

```

Con el conjunto de datos `X_train` entrenamos nuestro modelo SVM para clasificación usando el kernel lineal.

```

1 svm_clf = SVC(kernel='linear', C=1.0, random_state=42)
2 svm_clf.fit(X_train, y_train)

```



SVC

SVC(kernel='linear', random_state=42)

Una vez entrenado nuestro modelo usamos los datos de `X_test`, para checar la precisión del modelo.

Podemos calcular la precisión de nuestro modelo con los datos de prueba con la métrica *accuracy* que se define como:

$$\begin{aligned}
 accuracy &= \frac{\text{Todas las predicciones correctas}}{\text{Todas las predicciones}} \\
 &= \frac{\sum \text{Verdadero pos} + \text{Verdaderos neg}}{\sum \text{Verdadero pos} + \text{Falsos pos} + \text{Falsos neg} + \text{Verdaderos neg}}
 \end{aligned}$$

```

1 y_pred = svm_clf.predict(X_test)
2 accuracy = accuracy_score(y_test, y_pred)
3 print("Precision antes del envenenamiento:{:.2%}".format(accuracy))

```



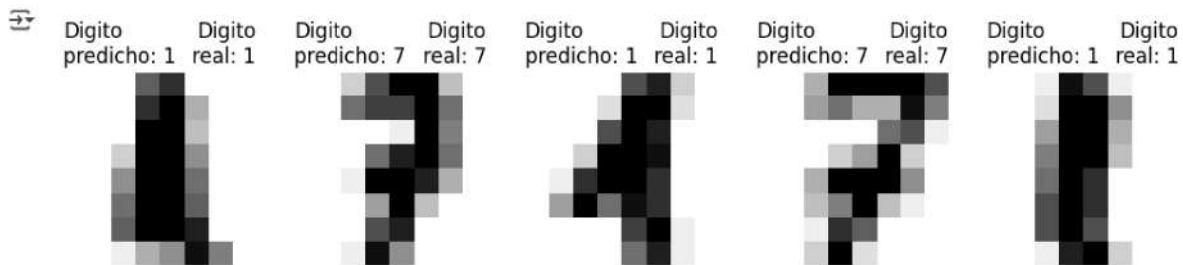
Presición antes del envenenamiento: 100.00%

Por ejemplo, si graficamos los primeros 5 dígitos, podemos ver que están correctamente clasificados.


```

1  _, axes = plt.subplots(nrows=1, ncols=6, figsize=(15, 3))
2  for ax, images, label, digito in zip(axes, X_train,
3      y_predict, y_train):
4      ax.set_axis_off()
5      images = images.reshape(8, 8)
6      ax.imshow(images, cmap=plt.cm.gray_r, interpolation='nearest')
7      ax.set_title('Digito\npredicho: ' + str(label), loc = "left")
8      ax.set_title('Digito\nreal: ' + str(digito), loc = "right")

```



5.1.1. Envenenamiento

Antes de iniciar el proceso de envenenamiento, es necesario convertir las etiquetas al formato one-hot encoding (codificación binaria). El siguiente código realiza esta transformación:

```

1  unique_classes = np.unique(y_train)
2  nb_samples = len(y_train)
3  nb_classes = len(unique_classes)
4  nb_samples_val=len(y_val)
5  print(unique_classes)
6  print(nb_samples)
7
8  (nb_samples, nb_classes)
9  y_train_ohe = np.zeros((nb_samples, nb_classes))
10 y_val_ohe=np.zeros((nb_samples_val, nb_classes))
11
12 for i, label in enumerate(y_train):
13     class_index = np.where(unique_classes == label)[0][0]

```

```

14     y_train_ohe[i, class_index] = 1
15     for i, label in enumerate(y_val):
16         class_index = np.where(unique_classes == label)[0][0]
17         y_val_ohe[i, class_index] = 1

```

Podemos envenenar el modelo utilizando los módulos *model.py*, *scikitlearn.py*, *attack.py*, *SVC.py* y *poisoning.py* disponibles en: <https://github.com/cheese-hub/SVM-Poisoning>

```

1 def get_adversarial_examples(
2     x_train,
3     y_train,
4     attack_idx,
5     x_val,
6     y_val,
7     kernel):
8     attack_classifier =
9     ScikitlearnClassifierSVC(
10        model=SVC(kernel=kernel),
11        clip_values=(0,255),
12        nb_classes=nb_classes)
13     attack_classifier.fit(x_train, y_train)
14     init_attack = np.copy(x_train[attack_idx])
15     y_attack = np.array([1, 1]) - np.copy(y_train[attack_idx])
16     attack = PoisoningAttackSVM(
17        attack_classifier, 0.01, 1.0, x_train,
18        y_train, x_val, y_val, max_iter=20)
19
20     x_attack, y_attack = attack.poison(
21        np.array([init_attack]),
22        y=np.array([y_attack]))
23     return x_attack, y_attack

```

Analicemos el caso en el que introducimos un único punto de envenenamiento al conjunto de entrenamiento.

```

1     num_poisoned_samples= 1
2     x_poison=X_train.copy()

```

```

3  y_poison=y_train.copy()
4  X_attack=[]
5  for attack_idx in range(num_poisoned_samples):
6      x_attackpoint,y_attackpoint = get_adversarial_examples(
7          X_train,
8          y_train_ohe,
9          attack_idx,
10         X_test,
11         y_val_ohe,
12         "linear")
13     x_poison=np.concatenate((x_attackpoint,x_poison),axis=0)
14     id_array = np.argmax(y_attackpoint, axis=1)
15     y_attackpoint=np.where(id_array == 0, 1, 7)
16     y_poison=np.concatenate((y_attackpoint,y_poison),axis=0)
17     X_attack.append(x_attackpoint)

```

➡ SVM poisoning:  1/? [00:00<00:00, 1.09it/s]

La siguiente imagen muestra una comparación entre el punto original y el punto modificado tras el proceso de envenenamiento. Cabe destacar que los cambios son pe-

```

array([ 0.,  0.,  0., 12., 13.,  5.,  0.,  0.,  0.,  0.,  0., 11., 16.,
        9.,  0.,  0.,  0.,  0.,  3., 15., 16.,  6.,  0.,  0.,  0.,  7.,
       15., 16., 16.,  2.,  0.,  0.,  0.,  0.,  1., 16., 16.,  3.,  0.,
        0.,  0.,  0.,  1., 16., 16.,  6.,  0.,  0.,  0.,  0.,  1., 16.,
       16.,  6.,  0.,  0.,  0.,  0.,  0., 11., 16., 10.,  0.,  0.])

```

(a) Punto antes del envenenamiento

```

[array([[0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 1.19982973e+01,
        1.29977843e+01, 4.99873082e+00, 0.00000000e+00, 0.00000000e+00,
        0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 1.09978826e+01,
        1.59974654e+01, 8.99826333e+00, 0.00000000e+00, 0.00000000e+00,
        0.00000000e+00, 0.00000000e+00, 2.99886757e+00, 1.49985036e+01,
        1.59979718e+01, 5.99842399e+00, 0.00000000e+00, 0.00000000e+00,
        0.00000000e+00, 6.99976675e+00, 1.49984873e+01, 1.59980154e+01,
        1.59975198e+01, 1.99830057e+00, 0.00000000e+00, 0.00000000e+00,
        0.00000000e+00, 0.00000000e+00, 9.98203663e-01, 1.59974925e+01,
        1.59973521e+01, 2.99848927e+00, 0.00000000e+00, 0.00000000e+00,
        0.00000000e+00, 0.00000000e+00, 9.98967475e-01, 1.59981490e+01,
        1.59975264e+01, 5.99881010e+00, 0.00000000e+00, 0.00000000e+00,
        0.00000000e+00, 0.00000000e+00, 9.99592840e-01, 1.59980228e+01,
        1.59978516e+01, 5.99893944e+00, 0.00000000e+00, 0.00000000e+00,
        0.00000000e+00, 1.68273875e-07, 0.00000000e+00, 1.09982519e+01,
        1.59983276e+01, 9.99893151e+00, 0.00000000e+00, 0.00000000e+00]])]

```

(b) Punto después del envenenamiento

queños, pero si aumentamos el número máximo de iteraciones así como el tamaño del paso, estos cambios son más notables. Para evaluar el impacto de este ataque, entrenaremos un nuevo modelo SVM con el dato envenenado y analizaremos su rendimiento.

```
1 svm_clf2 = SVC(kernel='linear', C=1.0, random_state=42)
2 svm_clf2.fit(x_poison, y_poison)
3 y_pred2 = svm_clf2.predict(X_test)
4 accuracy = accuracy_score(y_test, y_pred2)
5 print("Precision despues de 1 instancia de envenenamiento:
6 {:.2%}".format(accuracy))
```

Precision despues de 1 instancia de envenenamiento: 100.00%

Observemos que, con un único punto de envenenamiento, no logramos degradar el rendimiento del modelo. Para maximizar el impacto, podríamos incrementar el número de puntos envenenados, lo que reduciría aún más la precisión del SVM. Sin embargo, este enfoque presenta un desafío: el costo computacional.

Haciendo otra comparativa, ahora considerando 10 muestras envenenadas y entrenando nuevamente un SVM obtenemos el siguiente resultado.

```
1 num_poisoned_samples=10
2 x_poison=X_train.copy()
3 y_poison=y_train.copy()
4 X_attack=[]
5 for attack_idx in range(num_poisoned_samples):
6     x_attackpoint,y_attackpoint = get_adversarial_examples(
7     X_train,
8     y_train_ohe,
9     attack_idx,
10    X_test,
11    y_val_ohe,
12    "linear")
13 x_poison=np.concatenate((x_attackpoint,x_poison),axis=0)
14 id_array = np.argmax(y_attackpoint, axis=1)
15 y_attackpoint=np.where(id_array == 0, 1, 7)
16 y_poison=np.concatenate((y_attackpoint,y_poison),axis=0)
```

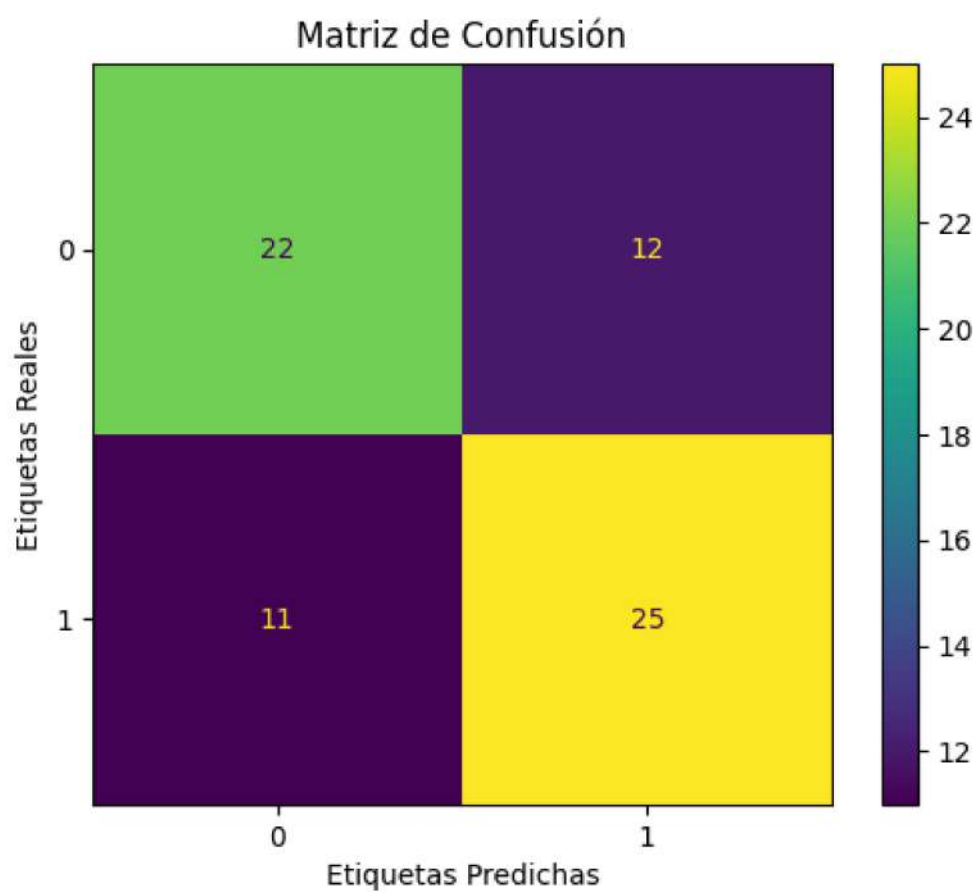
```
17 X_attack.append(x_attackpoint)
```

Evaluando el modelo obtenemos:

```
1 svm_clf3 = SVC(kernel='linear', C=1.0, random_state=42)
2 svm_clf3.fit(x_poison, y_poison)
3 y_pred3 = svm_clf3.predict(X_test)
4 accuracy3 = accuracy_score(y_test, y_pred3)
5 print("Precision despues de 1 instancia de envenenamiento:
6 {:.2%}".format(accuracy))
```

Precision despues del envenenamiento: 67.14%

De esta manera logramos reducir el rendimiento del modelo al 67.14% solamente inyectando 10 puntos de envenenamiento. Si imprimimos la matriz de confusión, podemos observar que hay 23 muestras clasificadas erróneamente de las 70 totales, por lo que nuestro envenenamiento tuvo éxito.



5.2. Aplicaciones: Wine

Para este apartado consideraremos los datos Wine disponibles en scikit-learn, se puede acceder a ellos importando la librería datasets, en conjunto con el siguiente código:

```
1 wine = load_wine()
```

Este dataset contiene 178 observaciones, con 13 características (Alcohol, Ácido málico, Ceniza, Alcalinidad de la ceniza, Magnesio, Fenoles totales, Flavonoides, Fenoles no flavonoides, Proantocianinas, Intensidad del color, Matiz, OD280/OD315 de vinos diluidos y Prolina); resultados de un análisis químico de vinos cultivados en una misma región de Italia por 3 productores diferentes.

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flavanoids	nonflavonoid_phenols	proanthocyanins	color_intensity	hue	od280/od315_of_diluted_wines	proline	etiquetas
0	14.23	1.71	2.43	15.6	127.0	2.80	3.06	0.28	2.29	5.64	1.04	3.92	1065.0	0
1	13.20	1.78	2.14	11.2	100.0	2.65	2.76	0.26	1.28	4.38	1.05	3.40	1050.0	0
2	13.16	2.36	2.67	18.6	101.0	2.80	3.24	0.30	2.81	5.68	1.03	3.17	1185.0	0
3	14.37	1.95	2.50	16.8	113.0	3.05	3.49	0.24	2.18	7.80	0.86	3.45	1480.0	0
4	13.24	2.59	2.87	21.0	118.0	2.80	2.69	0.39	1.82	4.32	1.04	2.93	735.0	0

Para este caso y con el fin de poder ver los resultados gráficamente consideramos únicamente dos características de las 13 disponibles solo para dos productores de vino, el segundo y el tercer productor. Con un análisis de componentes principales es fácil determinar que las características más representativas son *Flavonoides* y la *Intensidad del color*, las cuales consideraremos para el análisis.

```
1 X = np.c_[wine.data[wine.target != 2, 6], wine.data[wine.target != 2, 9]]
2 y = 2*wine.target[wine.target != 2]-1
```

Primero empezamos dividiendo los conjuntos en entrenamiento, prueba y validación.

```
1 X, X_train, y, y_train = train_test_split(X, y,
2     test_size=0.23, random_state=42)
3 X_test, X_val, y_test, y_val = train_test_split(X, y,
4     test_size=0.5, random_state=42)
```

Cada conjunto con tamaños

```
1 print(f'Dimension del conjunto de entrenamiento:{X_train.shape[0]}')
2 print(f'Dimension del conjunto de prueba:{X_test.shape[0]}')
```

```
3 print(f'Dimension del conjunto de validacion:{X_val.shape[0]}')
```

⇒ Tamaño del conjunto de entrenamiento: 30
 Tamaño del conjunto de prueba: 50
 Tamaño del conjunto de validación: 50

Luego entrenamos un modelo SVM usando el módulo de scikit-learn con un kernel lineal.

```
1 svm = SVC(kernel='linear', C=1)
2 svm.fit(X_train, y_train)
```

Posteriormente obtenemos la precisión antes del ataque

```
1 y_pred = svm.predict(X_test)
2 accuracy_before_attack = accuracy_score(y_test, y_pred)
3 print("Precision antes del ataque:
4 {:.2%}".format(accuracy_before_attack))
```

5.2.1. Envenenamiento

Ahora procedemos a envenenar el modelo, pero antes haremos un cambio en las etiquetas para presentarlas en un formato One-hot encode, con la siguiente sección de código.

```
1 unique_classes = np.unique(y_train)
2 nb_samples = len(y_train)
3 nb_classes = len(unique_classes)
4 nb_samples_val=len(y_val)
5 print(unique_classes)
6 print(nb_samples)
7
8 y_train_ohe = np.zeros((nb_samples, nb_classes))
9 y_val_ohe=np.zeros((nb_samples_val, nb_classes))
```

⇒ SVC
 SVC(C=1, kernel='linear')

 Precisión antes del ataque: 92.00%

```

10
11     for i, label in enumerate(y_train):
12         class_index = np.where(unique_classes == label)[0][0]
13         y_train_ohe[i, class_index] = 1
14     for i, label in enumerate(y_val):
15         class_index = np.where(unique_classes == label)[0][0]
16         y_val_ohe[i, class_index] = 1

```

Para envenenar el modelo usamos los módulos mencionados anteriormente y disponibles en <https://github.com/cheese-hub/SVM-Poisoning>

```

1     def get_adversarial_examples(x_train, y_train,
2                                 attack_idx, x_val, y_val, kernel):
3         attack_classifier = ScikitlearnClassifierSVC(
4             model=SVC(kernel=kernel),
5             clip_values=(0,20),
6             nb_classes=nb_classes)
7         attack_classifier.fit(x_train, y_train)
8         init_attack = np.copy(x_train[attack_idx])
9         y_attack = np.array([1, 1]) - np.copy(y_train[attack_idx])
10        attack = PoisoningAttackSVM(
11            attack_classifier, 0.01, 1.0,
12            x_train, y_train,
13            x_val, y_val, max_iter=100)
14
15        x_attack, y_attack = attack.poison(np.array([init_attack]),
16                                           y=np.array([y_attack]))
17        return x_attack, y_attack

```

Primero vamos a agregar 1 solo punto de envenenamiento.

```

1     num_poisoned_samples=1
2     x_poison=X_train
3     y_poison=y_train
4     X_attack=[]
5     y_attack=[]

```

```

6     for attack_idx in range(num_poisoned_samples):
7         x_attackpoint, y_attackpoint = get_adversarial_examples(X_train,
8             y_train_ohe, attack_idx, X_test, y_val_ohe, "linear")
9         x_poison=np.concatenate((x_attackpoint,x_poison),axis=0)
10        id_array = np.argmax(y_attackpoint, axis=1)
11        y_attackpoint=np.where(id_array == 0,-1, 1)
12        y_poison=np.concatenate((y_attackpoint,y_poison),axis=0)
13        X_attack.append(x_attackpoint)
14        y_attack.append(y_attackpoint)

```

 SVM poisoning:  1/? [00:00<00:00, 2.05it/s]

Veamos como se compara con el punto original, con el punto envenenado.

 `array([2.78, 6.25])`  `[array([[2.80417424, 6.29376764]])]`

(a) Datos antes del envenenamiento

(b) Datos después del envenenamiento

Nuevamente entrenamos otro SVM con los datos envenenados y obtenemos su precisión.

```

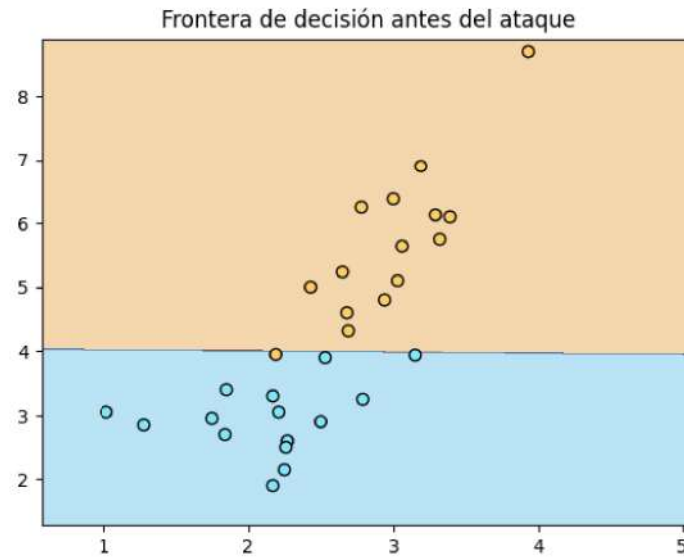
1     svm_clf2 = SVC(kernel='linear', C=1.0, random_state=42)
2
3     svm_clf2.fit(x_poison, y_poison)
4
5     y_pred2 = svm_clf2.predict(X_test)
6     accuracy2 = accuracy_score(y_test, y_pred2)
7     print("Precisi n despues de 1 punto de ataque: {:.2%}".format(accuracy2))

```

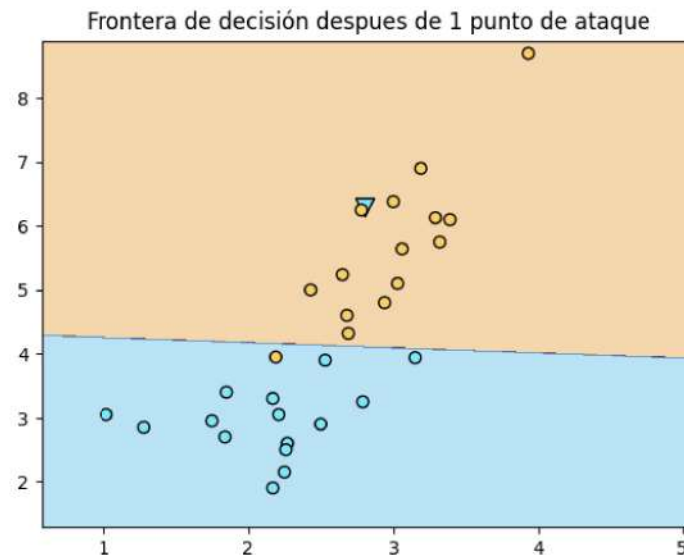
 Precisión despues de 1 puntos de ataque: 92.00%

Si bien con esto no logramos disminuir el rendimiento del modelo, sí logramos mover los coeficientes. Por ejemplo, al comparar los hiperplanos separadores del modelo antes y después del envenenamiento, podemos ver cómo este se desplaza ligeramente hacia arriba.

Los triángulos en los diagramas representan los puntos envenenados.



(a) Hiperplano antes del envenenamiento, los coeficientes que definen este hiperplano son $\beta_0 = 6.58794975$, $\beta_1 = -0.03$ y $\beta_2 = -1.63$



(b) Hiperplano después de 1 punto de envenenamiento, los coeficientes que definen este hiperplano son $\beta_0 = 5.03834843$, $\beta_1 = -0.09367499$ y $\beta_2 = -1.16214629$

Por otra parte, si aumentamos la cantidad de puntos de ataque a 5, podemos degradar el rendimiento del modelo en un 6 %.

⇄ SVM poisoning: ■ 1/? [00:00<00:00, 2.84it/s]
 SVM poisoning: ■ 1/? [00:00<00:00, 2.71it/s]
 SVM poisoning: ■ 1/? [00:00<00:00, 3.07it/s]
 SVM poisoning: ■ 1/? [00:00<00:00, 2.85it/s]
 SVM poisoning: ■ 1/? [00:00<00:00, 2.54it/s]

```
1 svm_clf3 = SVC(kernel='linear', C=1.0, random_state=42)
2
3 svm_clf3.fit(x_poison, y_poison)
4
5 y_pred3 = svm_clf3.predict(X_test)
6 accuracy3 = accuracy_score(y_test, y_pred3)
7 print("Precisi n despues de 5 puntos de ataque:
8 {:.2%}".format(accuracy3))
```

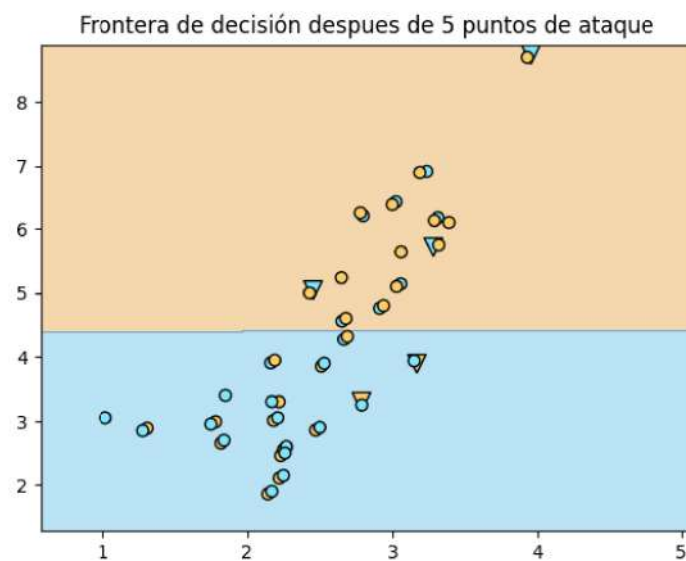
Si nuevamente aumentamos la cantidad a 25 puntos de envenenamiento,

```
1 svm_clf4 = SVC(kernel='linear', C=1.0, random_state=42)
2
3 svm_clf4.fit(x_poison, y_poison)
4
5 y_pred4 = svm_clf4.predict(X_test)
6 accuracy4 = accuracy_score(y_test, y_pred4)
7 print("Precisi n despues de 25 punto de ataque:
8 {:.2%}".format(accuracy4))
```

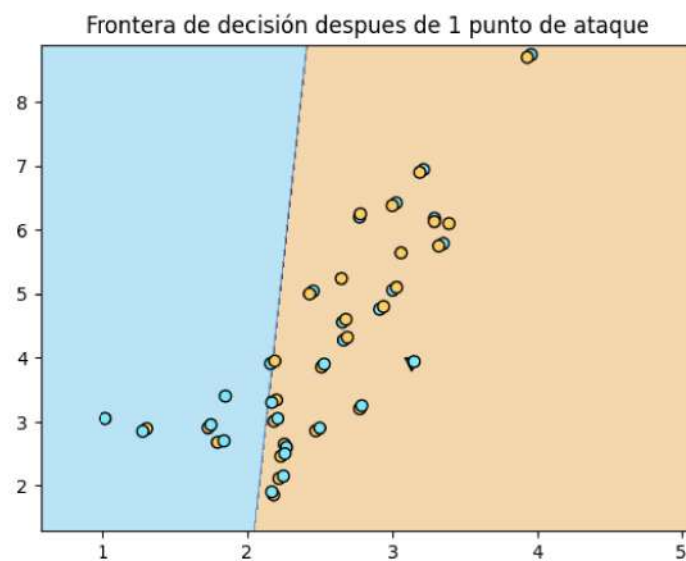
Con esto podemos observar cómo el rendimiento decae al 76 %, en la siguiente imagen se muestra cómo cambia la frontera de decisión drásticamente al aumentar la cantidad de puntos de envenenamiento.

⇄ Precisión despues de 5 puntos de ataque: 86.00%

Precisión despues de 25 punto de ataque: 76.00%



(a) Hiperplano despues de 5 puntos de envenenamiento, los coeficientes que definen este hiperplano son $\beta_0 = 3.25937224$, $\beta_1 = -0.000217515019$ y $\beta_2 = -0.740648859$



(b) Hiperplano despues de 25 puntos de envenenamiento, los coeficientes que definen este hiperplano son $\beta_0 = 1.78647242$, $\beta_1 = -0.8989058$ y $\beta_2 = 0.04275622$

A partir de estos dos ejemplos ilustramos cómo un posible adversario puede reducir significativamente el desempeño del modelo.

En particular, en el primer caso logramos reducir el rendimiento en un 30 %; mientras que en el segundo, la incorporación de 25 puntos adversariales, lograron reducir el rendimiento en un 16 %. No obstante, más relevante que evidenciar la vulnerabilidad del modelo es analizar las estrategias de defensa que pueden emplearse para mitigar estos ataques. En la siguiente sección, abordamos este problema y exploramos posibles soluciones.

Capítulo 6

Metodos de defensa y conclusiones

Hasta el momento hemos descrito cómo envenenar una máquina de soporte vectorial al intentar degradar el rendimiento del modelo, generando puntos que aumentan la pérdida. Sin embargo, más relevante que las técnicas de envenenamiento en sí mismas, es qué estrategias pueden emplearse para mitigar estas amenazas de seguridad, no solo en SVM, sino en diferentes modelos de aprendizaje automático.

Esto es de vital importancia, debido al creciente uso de modelos de machine learning en los últimos años. Por ejemplo, en el artículo BadNets de Tianyu et al. (2019), se muestra cómo se puede envenenar un conjunto limpio de imágenes de señales de tránsito introduciendo píxeles amarillos en forma de patrones específicos, acompañados de cambios en las etiquetas. Los resultados experimentales demostraron una reducción significativa en la precisión de los clasificadores entrenados con este conjunto de imágenes contaminadas, al utilizar los patrones de píxeles amarillos como activadores del ataque. En cuestiones como conducción autónoma, este tipo de ataques puede ser crucial.

En la presente sección, presentamos algunas estrategias para mitigar este tipo de ataques. Cabe destacar que este problema es complejo y no existe un marco unificado que garantice la defensa contra envenenamiento, ya sea para SVM u otros modelos de aprendizaje automático. Presentamos algunas estrategias; para un estudio más detallado se pueden consultar en la bibliografía. Las principales propuestas comparten un enfoque común: realizar un prefiltrado de los datos para identificar y eliminar aquellos datos considerados corruptos o comprometidos, combinándolo, en algunos casos, con



Figura 6.1: La imagen fue extraída del artículo de BadNets y muestra un ataque de envenenamiento usando patrones de píxeles amarillos.

el entrenamiento de modelos más robustos.

6.1. Metodos de defensa

6.1.1. Detección de Valores Atípicos

La primera propuesta es presentada por Paudice, A. et al. (2018) y busca mitigar los efectos de los ataques de envenenamiento mediante la detección de valores atípicos, haciendo un prefiltrado de estos en el conjunto de entrenamiento. Empíricamente se ha demostrado que los puntos generados por ataques de envenenamiento tienen diferente distribución de los datos genuinos, debido a que al momento de su creación no se consideran restricciones en las características para evitar detectar los puntos adversariales, lo que genera valores atípicos.

El procedimiento es el siguiente:

1. Se selecciona un subconjunto confiable de datos y luego se segmenta por clases.
2. Se entrena un detector de valores atípicos basado en distancias dentro de cada clase, obteniendo una puntuación de anormalidad para cada dato x .
3. Se calcula un percentil α mediante una función de distribución acumulativa empírica, que define el umbral de separación entre datos genuinos y adversariales.
4. Los puntos con anormalidad superior al umbral se consideran atípicos y se eliminan del conjunto de datos.

Es importante señalar que este mecanismo solo funciona si los datos usados para entrenar el detector de valores atípicos son confiables; si este conjunto está comprometido, el detector también puede ser envenenado. La principal desventaja de este enfoque es que es más difícil detectar envenenamientos con cambio de etiquetas, aunque usualmente este tipo de ataque suele ser menos efectivo.

6.1.2. Procedencia de Datos

Esta propuesta, descrita por Baracaldo, N. et al. (2017), propone utilizar la procedencia de los datos (metadatos) como una forma de identificar y filtrar puntos de datos maliciosos, dado que muchos modelos recopilan información en línea y es posible conocer la procedencia de los datos. Este método introduce el concepto de *firma de procedencia*, el cual es un identificador de la fuente específica de la que provienen los datos (cuenta o usuario).

Este método propone dos enfoques:

- Datos parcialmente confiables:
 1. Primero, dividimos el conjunto de datos en dos partes: confiable y no confiable exteriormente.
 2. Para cada punto en el conjunto no confiable, se hace un registro de su procedencia que describe su origen.
 3. Los datos no confiables se segmentan para que cada segmento comparta la misma firma de procedencia.
 4. Se entrenan dos modelos: uno con todos los datos y el otro con todos los datos menos el segmento en cuestión.
 5. Finalmente, se compara el desempeño de ambos modelos en el conjunto confiable. Si el modelo entrenado sin el segmento tiene mejor desempeño, entonces este segmento se clasifica como envenenado y se elimina del conjunto de datos no confiables.
- Datos completamente desconfiables:

1. Primero, se agrupan los datos por firma de procedencia y se dividen aleatoriamente en subconjuntos de entrenamiento y evaluación.
2. Para cada firma, se entrenan dos modelos: uno con todos los datos y otro sin los datos de la firma seleccionada.
3. Posteriormente, se evalúan los modelos en el conjunto de evaluación, excluyendo los datos del segmento correspondiente.
4. Si el modelo sin el segmento correspondiente supera en desempeño al modelo con el segmento, se elimina esa firma del conjunto de entrenamiento y evaluación.

6.1.3. Modelo De-Pois

Este enfoque es propuesto por Chen, J. et al. (2021) y es similar a los anteriores, pues filtra muestras que considera envenenadas. Asume, pues, que las muestras envenenadas presentan mayores discrepancias con las predicciones en comparación con la distribución de los datos limpios, y estas discrepancias se pueden medir mediante las diferencias con un modelo imitador. Otra suposición importante que se hace es que se considera que contamos con un conjunto de datos limpios.

El modelo consiste en lo siguiente:

1. Primero, emplea un modelo condicional generativo adversarial (cGAN) para generar datos sintéticos con la misma distribución que los datos limpios. Usa un autenticador para supervisar el proceso y asegurar la calidad de los datos *aumentados* sintéticos.
2. Posteriormente, hace uso de un modelo generativo adversarial con gradiente penalizado (WGAN-GP) para aprender la distribución de los datos aumentados.
3. Una vez completado el entrenamiento, se utiliza el discriminador de WGAN-GP como el modelo imitador; este imita el comportamiento de predicción de un modelo entrenado con datos limpios.

4. Una vez que tenemos el modelo imitador, se establece un límite de detección, en el cual, si la salida del modelo imitador para una muestra está por debajo del límite de detección, esa muestra se clasifica como envenenada.

De esta forma, De-Pois se usa como un filtro previo al entrenamiento de modelos de aprendizaje automático, asegurando que las muestras envenenadas no afecten al modelo entrenado.

6.1.4. Modelo TRIM

Este modelo es presentado por Jagielski, M. et al. (2021) y busca eliminar iterativamente las muestras sospechosas del conjunto de entrenamiento y ajustar el modelo en el subconjunto restante. El esquema es el siguiente:

1. Ajustar inicialmente un modelo con todos los datos.
2. Evaluar el error de cada predicción de cada muestra e identificar el subconjunto de muestras cuyo error es más alto, ya que estas son más probables de estar contaminadas.
3. Luego, reentrenar el modelo usando las muestras restantes y repetir este proceso hasta alcanzar la convergencia, es decir, cuando las muestras identificadas como sospechosas no cambian significativamente entre iteraciones.

La principal limitación de este modelo es la tendencia a encontrar mínimos locales en lugar de mínimos globales, especialmente si la proporción de datos contaminados es alta.

Las estrategias presentadas anteriormente son algunas de las que podemos emplear para disminuir el impacto del atacante; sin embargo, no son las únicas: existen otras como DUTI, CD, Sever, etc. Pero todas se basan en el mismo principio de hacer un prefiltrado de los datos.

Por otro lado, las investigaciones enfocadas en cómo fortalecer la robustez de los modelos son relativamente más escasas. Sin embargo, algunas estrategias destacan el

uso de técnicas de validación cruzada en subconjuntos de datos limpios, lo que permite evaluar el impacto de las muestras sospechosas en el rendimiento del modelo. Otra estrategia consiste en diseñar modelos que sean menos sensibles a datos ruidosos. Esto puede lograrse mediante la adición de ruido calibrado a través de técnicas de suavizado aleatorio, reduciendo el peso de las muestras con mayores pérdidas, un enfoque conocido como robustez certificada. Adicionalmente, se pueden incorporar restricciones adicionales al modelo para evitar un sobreajuste a patrones de envenenamiento, penalizando los pesos que se alineen excesivamente con características específicas.

Por ejemplo, una propuesta de un modelo de SVM más robusto es presentada por Weerasinghe et al. (2011) en su artículo *Defending Support Vector Machines Against Data Poisoning Attacks*, en el cual mejora el SVM para que sea resistente ante ataques de envenenamiento de inversión de etiquetas. Funciona de la siguiente forma:

1. Se introduce un modelo SVM ponderado que incluye el concepto de *K-LID* (*Kernel-Based Local Intrinsic Dimensionality*). Esta métrica identifica muestras atípicas midiendo qué tan aislada está una muestra en su vecindad local usando la distancia del *kernel*, permitiendo calcular distancias en espacios de alta dimensionalidad. Muestras con valores anómalos de K-LID se consideran posibles datos envenenados.
2. Luego se desarrollan 3 variaciones específicas de K-LID:
 - a) **K-LID-P**: Detecta si una muestra etiquetada como positiva se comporta de manera anómala respecto a otras muestras positivas.
 - b) **K-LID-N**: Identifica muestras etiquetadas como negativas con comportamiento anómalo respecto a muestras negativas.
 - c) **K-LID**: Combina las contribuciones de las dos métricas anteriores. Esto sirve para identificar casos en los que la muestra envenenada está afectando a ambas clases. Por ejemplo, una muestra con etiqueta positiva que está muy cercana a muestras negativas tendrá un comportamiento sospechoso.

Estas variaciones ayudan a identificar muestras envenenadas según cómo impactan las etiquetas en la distribución de los datos.

3. Posteriormente se asignan pesos a las muestras en función de los valores de K-LID, penalizando más a las muestras que se consideran sospechosas y menos a las más confiables de acuerdo con la métrica.

De esta forma, logramos modificar la frontera de decisión del SVM para que sea menos sensible a los datos envenenados; esto nos permite lograr un SVM más robusto frente a manipulaciones adversarias.

La implementación de técnicas de detección de envenenamiento en el entrenamiento de modelos incrementa su capacidad de resistencia ante datos contaminados, lo que favorece el desarrollo de sistemas con mayor inmunidad. Esto genera un debate sobre el aumento de los costos involucrados en el proceso de envenenamiento de datos; se espera que sea más costoso en los próximos años, lo cual podría incluso desalentar futuros intentos de ataques desde el principio. Una práctica común es entrenar un modelo de aprendizaje automático que imite las características del modelo objetivo, lo cual permite aproximarse lo más posible a las superficies de separación del clasificador y, a su vez, probar el efecto del ataque. Pero esto genera que los ataques tengan que aumentar los recursos disponibles; sobre todo en el uso de hardware avanzado como GPUs. Los recursos del atacante, vistos como un “presupuesto”, son factores que hay que tener en cuenta en posibles ataques, ya que se prevé un aumento de estos en los próximos años; además, los recursos disponibles determinan si los atacantes se limitan a modelos lineales o abordan estructuras más complejas.

Es importante resaltar que no existe, hasta el momento, un marco unificado que defina trayectorias claras para proteger de manera efectiva las máquinas de soporte vectorial o, en general, cualquier modelo de machine learning. Este campo permanece en constante evolución, ya que se desarrollan continuamente métodos más sofisticados para comprometer el rendimiento de los modelos, mientras que, a la par, surgen estrategias adaptativas para contrarrestar estos ataques.

6.2. Conclusiones

Los ingenieros y científicos de datos deben tener siempre presente la amenaza latente de adversarios que buscan comprometer los sistemas, explotando vulnerabilidades en las etapas de vida de un modelo de machine learning.

Por esta razón, al entrenar un modelo, es fundamental considerar las estrategias que se consideren más adecuadas para protegerlo. En la práctica, lo ideal es combinar un prefiltrado de datos junto con el diseño de un modelo más robusto. Este enfoque puede ofrecer una mayor confianza en las predicciones generadas por el modelo. Si bien estas estrategias han demostrado ser eficaces, aún existen numerosos aspectos por explorar para garantizar que los modelos no solo sean seguros, sino también eficientes y escalables a grandes volúmenes de datos.

Por último, es vital reconocer que la seguridad no es un estado final, sino un proceso continuo. Incluso modelos actualmente "seguros" podrían volverse vulnerables ante nuevos tipos de ataques o cambios en la distribución de los datos. Innovaciones como la integración de sistemas de detección en tiempo real, el uso de entornos de ejecución seguros o la aplicación de técnicas basadas en blockchain podrían complementar estos enfoques tradicionales. La investigación continua en este campo permitirá que el machine learning avance hacia un estado más eficiente, confiable y seguro, adaptable a las necesidades de la sociedad moderna.

Apéndice A

SVM multiclase

Originalmente, el problema de SVM surgió para una clasificación binaria y es un área que, a día de hoy, sigue en investigación. Podemos tomar dos enfoques para un SVM multiclase: uno es mediante la construcción y combinación de varios clasificadores binarios, y el otro consiste en la formulación de un solo problema de optimización. Describiremos a grandes rasgos estos enfoques.

Método indirecto

Uno vs Todos (one-vs-all, OVA). Supongamos que tenemos un problema de clasificación con k clases diferentes. La idea es construir k modelos, lo que nos genera k hiperplanos separadores. En donde el m -ésimo clasificador binario se entrena considerando los ejemplos de entrenamiento de la m -ésima clase como positivos y los de las $k - 1$ clases restantes se consideran como clases negativas. Entonces, para clasificar una nueva observación x^* , observamos el valor máximo de cada clasificador $f_i(x^*)$ con $i = 1, \dots, k$, por lo que este punto se agrupa en la clase del máximo valor del clasificador. Los principales problemas de este enfoque son que los conjuntos están desequilibrados y que hace falta entrenar k problemas cuadráticos, cada uno con el número de variables igual al número de variables de entrenamiento.

Todos vs Todos (all-vs-all, AVA). Este método consiste en construir todos los clasificadores binarios posibles entre una clase y las restantes. Por lo tanto, se generan $\binom{k}{2} = \frac{k(k-1)}{2}$. Una vez contruidos todos los clasificadores, una nueva observación x^* queda determinada evaluando en cada clasificador la clase de x^* y asignándole un voto a esta. Por lo que x^* estará en la clase que tenga más votos. También en este enfoque

existen algunos problemas, por ejemplo, el número de clasificadores es mayor que en el anterior. La ventaja es que el número de variables en el problema dual de optimización cuadrática es mucho menor, ya que solo estamos considerando dos clases a la vez.

Desde el punto de vista computacional, para AVA se aplica un entrenamiento mayor de clasificadores, mientras que OVA requiere un número lineal de clasificadores. Sin embargo, AVA tiene problemas de optimización más pequeños, lo que hace que este enfoque sea preferible.

Metodo directo.

Este segundo enfoque consiste en diseñar una única función que entrene las k SVMs binarias simultáneamente y maximice los márgenes de cada clase, la idea es similar del método uno contra todos. Se construyen K modelos de 2 clases donde la función m -ésima separa los vectores de entrenamiento de la clase m de los otros vectores por lo tanto hay k funciones de decisión, la formulación del problema es

$$\begin{aligned} \min_{\beta, b, \xi} \quad & \frac{1}{2} \sum_{m=1}^k \beta_m^T \beta_m + C \sum_{i=1}^l \sum_{m \neq y_i} \xi_{m,i} \\ \text{subject to:} \quad & \beta_{y_i}^T \phi(x_i) + b_{y_i} \geq \beta_m^T \phi(x_i) + b_m + 2 - \xi_i^m, \\ & \xi_i^m \geq 0, \quad i = 1, \dots, l, \quad m \in \{1, \dots, k\} \setminus y_i. \end{aligned}$$

Donde $\xi_{m,i}$ es el valor m, i de la matriz de variables de holgura, ϕ es la función que mapea al espacio de características y C es el parametro de penalización o Costo.

Entonces la función de decisión es:

$$\arg \max_{m=1, \dots, k} (w_m^T \phi(x) + b_m).$$

El principal problema con este método es el tiempo computacional necesario para resolver la gran cantidad de variables en el problema de optimización cuadrática. Existen otros métodos para la clasificación de un SVM multiclase que atacan el principal problema de la gran cantidad de variables para optimizar; sin embargo, son líneas que siguen en investigación.

Apéndice B

Aprendizaje incremental de SVM

En esta breve sección hablaremos de forma muy general sobre el SVM incremental, presentado por Poggio y Cauwenberghs. Dado que el entrenamiento del SVM es la tarea más complicada desde el punto de vista computacional, se desarrollaron nuevos métodos para usar un modelo existente. Uno de estos métodos es el aprendizaje incremental, que, a grandes rasgos, es un proceso que nos permite volver a entrenar un SVM añadiendo un nuevo punto sin necesidad de calcular nuevamente todo el modelo desde cero, utilizando el modelo entrenado previamente (otro modelo que usa el SVM previamente entrenado es el entrenamiento por lotes, en el que el SVM se entrena constantemente añadiendo nuevos lotes a los datos de entrenamiento).

Este método puede ser crucial, sobre todo cuando trabajamos en aplicaciones prácticas, como el aprendizaje en línea (cuando constantemente recibimos nuevos datos y es necesario actualizar el modelo). La clave para el problema es mantener las condiciones de Kuhn-Tucker mientras se agrega un nuevo punto a la solución. Como mencionamos en la sección anterior, estas condiciones dividen al conjunto de puntos en tres subconjuntos: S, E y R.

Cuando agregamos un nuevo punto x_c , su peso α_c se establece inicialmente en 0. Si este punto debería convertirse en un vector de soporte, entonces los pesos de otros puntos deben actualizarse para obtener una solución óptima, preservando las condiciones de Kuhn-Tucker. Las técnicas presentadas para obtener cómo cambian los vectores de soporte y la estructura de los conjuntos S, E y R de acuerdo a lo que denominan coeficientes de sensibilidad y sensibilidad de margen son presentadas por

Poggio en el artículo *Incremental and Decremental Support Vector Machine Learning*, que resumen en el siguiente algoritmo.

Consideremos $D'_{tr} = D_{tr} \cap (x_c, y_c)$, entonces la nueva solución $\{\alpha_i^{l+1}, b^{l+1}\}$ para $i = 1, 2, \dots, l+1$ queda expresado en términos de la presente solución $\{\alpha_i^l, b^l\}$ la inversa de la matriz jacobiana $\mathcal{R}$ y el candidato (x_c, y_c)

Algorithm 2 Aprendizaje incremental.

- 1: Inicializar α_c con cero.
 - 2: Si $g_c > 0$ entonces se termina el algoritmo pues c no es un vector de margen o de error.
 - 3: Si $g_c \leq 0$ aumentamos el mayor incremento posible a α_c hasta que se produzca una de las siguientes condiciones.
 - 4: $g_c = 0$: añadir c al conjunto de márgenes S , actualizar $\mathcal{R}$ y terminar.
 - 5: $\alpha_c = 0$: añadir c al conjunto E y terminar.
 - 6: Los elementos de D_{tr} se intercambian entre los conjuntos S, E y R de acuerdo a bookkeeping, luego actualizar la pertenencia de los elementos a los conjuntos y si S cambia actualizar $\mathcal{R}$
 - 7: Repetir si es necesario.
-

Bibliografía

- Baracaldo, N., Chen, B., Ludwig, H., & Safavi, J. A. (2017). Mitigating Poisoning Attacks on Machine Learning Models: A Data Provenance Based Approach. *Proceedings of the 10th ACM Workshop on Artificial Intelligence and Security*. <https://doi.org/10.1145/3128572.3140447>
- Bazaraa, M. S., Sherali, H. D., & Shetty, C. M. (2006). *Nonlinear Programming: Theory and Algorithms* (3rd). Wiley-Interscience.
- Biggio, B., Nelson, B., & Laskov, P. (2013). Poisoning Attacks Against Support Vector Machines. <https://arxiv.org/abs/1206.6389>
- Carmona Suárez, E. J. (2016). Tutorial sobre máquinas de vectores soporte (SVM).
- Cauwenberghs, G., & Poggio, T. (2000). Incremental and Decremental Support Vector Machine Learning. *Advances in Neural Information Processing Systems*, 13, 409-415. https://proceedings.neurips.cc/paper_files/paper/2000/file/155fa09596c7e18e50b58eb7e0c6ccb4-Paper.pdf
- Chen, H., & Koushanfar, F. (2023). Tutorial: Toward Robust Deep Learning Against Poisoning Attacks. *ACM Transactions on Embedded Computing Systems*, 22(3), Article 42, 1-15. <https://doi.org/10.1145/3574159>
- Chen, J., Zhang, X., Zhang, R., Wang, C., & Liu, L. (2021, mayo). De-Pois: An Attack-Agnostic Defense Against Data Poisoning Attacks. <https://arxiv.org/abs/2105.03592v1>
- Cinà, A. E., Grosse, K., Demontis, A., Biggio, B., Roli, F., & Pelillo, M. (2024, marzo). Machine Learning Security Against Data Poisoning: Are We There Yet? [University of Cagliari, Italy]. <https://arxiv.org/abs/2204.05986v3>

- Cortes, C., & Vapnik, V. (1995). Support-Vector Networks. *Machine Learning*, 20(3), 273-297. <https://doi.org/10.1007/BF00994018>
- Crammer, K., & Singer, Y. (2001). On the Algorithmic Implementation of Multi-class Kernel-Based Vector Machines. *Journal of Machine Learning Research*, 2, 265-292.
- Demontis, A., Melis, M., Pintor, M., Jagielski, M., Biggio, B., Oprea, A., Nita-Rotaru, C., & Roli, F. (2019). Why Do Adversarial Attacks Transfer? Explaining Transferability of Evasion and Poisoning Attacks. *Proceedings of the 28th USENIX Security Symposium*. <https://www.usenix.org/conference/usenixsecurity19/presentation/demontis>
- Dresden, T. U. (2014, enero). Machine learning: Hinge loss.
- Evgeniou, T., Pontil, M., & Poggio, T. (2000). *Regularization Networks and Support Vector Machines* (inf. téc.). Center for Biological y Computational Learning, Artificial Intelligence Laboratory, Massachusetts Institute of Technology. Cambridge, MA. <https://www.researchgate.net/publication/220391260>
- Fletcher, R. (1987). *Practical Methods of Optimization: Volume 2, Constrained Optimization*. John Wiley & Sons.
- Gu, T., Dolan-Gavitt, B., & Garg, S. (2019). BadNets: Identifying Vulnerabilities in the Machine Learning Model Supply Chain. <https://arxiv.org/abs/1708.06733>
- Hastie, T., Rosset, S., Tibshirani, R., & Zhu, J. (2004). The Entire Regularization Path for the Support Vector Machine. *Journal of Machine Learning Research*, 5, 1391-1415. <https://jmlr.csail.mit.edu/papers/volume5/hastie04a/hastie04a.pdf>
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). Support Vector Machines and Flexible Discriminants. En *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd, pp. 417-455). Springer. <https://doi.org/10.1007/978-0-387-84858-7>
- Hofmann, T., Schölkopf, B., & Smola, A. J. (2008). Kernel Methods in Machine Learning. *The Annals of Statistics*, 36(3), 1171-1220. <https://doi.org/10.1214/0090536070000000677>

- Hornik, K., Karatzoglou, A., & Meyer, D. (2006). Support Vector Machines in R. *Journal of Statistical Software*, 15(9), 1-28. <https://doi.org/10.18637/jss.v015.i09>
- Hsu, C.-W., & Lin, C.-J. (s.f.). A Comparison of Methods for Multi-Class Support Vector Machines.
- Jagielski, M., Oprea, A., Biggio, B., Liu, C., Nita-Rotaru, C., & Li, B. (2021, septiembre). Manipulating Machine Learning: Poisoning Attacks and Countermeasures for Regression Learning. <https://arxiv.org/abs/1804.00308v3>
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning: With Applications in R*. Springer. <https://doi.org/10.1007/978-1-4614-7138-7>
- Laskov, P., Gehl, C., Krüger, S., & Müller, K.-R. (2006). *Incremental Support Vector Learning: Analysis, Implementation and Applications* (inf. téc.). Fraunhofer-FIRST.IDA, Kekuléstraße 7, 12489 Berlin, Germany; University of Potsdam, August-Bebel-Straße 89, 14482 Potsdam, Germany.
- León C., E. (s.f.). Introducción a las máquinas de vector soporte (SVM) en aprendizaje supervisado.
- Mello, R. F., & Ponti, M. A. (2018). *Machine Learning: A Practical Approach on the Statistical Learning Theory*. Springer. <https://doi.org/10.1007/978-3-319-94989-5>
- Michie, D., Spiegelhalter, D. J., & Taylor, C. C. (Eds.). (1994). *Machine Learning, Neural and Statistical Classification*. Ellis Horwood.
- Morales, E., & Escalante, H. J. (2019). Máquinas de Soporte Vectorial. <https://ccc.inaoep.mx/~emorales/Cursos/NvoAprend/Acetatos/svm2019.pdf>
- Nazir, T., Qi, X., & Silvestrov, S. (2016). Linear and nonlinear classifiers of data with support vector machines and generalized support vector machines. En S. Silvestrov & M. Rančić (Eds.), *Engineering Mathematics II: Algebraic, stochastic and analysis structures for networks, data classification and optimization* (pp. 377-396). Springer. https://doi.org/10.1007/978-3-319-42105-6_18

- Paudice, A., Muñoz-González, L., Gyorgy, A., & Lupu, E. C. (2018, febrero). Detection of adversarial training examples in poisoning attacks through anomaly detection. <https://arxiv.org/pdf/1802.03041v1>
- Pothukuchi, S., Lambert, S., & Kalyanam, R. (2023, julio). SVM-Poisoning. <https://github.com/cheese-hub/SVM-Poisoning>
- Ramirez, M. A., Kim, S.-K., Al Hamadi, H., Damiani, E., Byon, Y.-J., Kim, T.-Y., Cho, C.-S., & Yeun, C. Y. (2022, febrero). Poisoning attacks and defenses on artificial intelligence: A survey [arXiv:2202.10276v2 [cs.CR]]. <https://arxiv.org/abs/2202.10276v2>
- Ramos Ponce, O. E. (2020). Support Vector Machines (SVM) - Parte 2.
- Resendiz Trejo, J. A. (2006). *Las máquinas de vectores de soporte para identificación en línea* [Tesis doctoral, Centro de Investigación y de Estudios Avanzados del Instituto Politécnico Nacional]. <https://tesis.ipn.mx/jspui/bitstream/123456789/24801/1/T1885.pdf>
- Rojas Rojas, J. A. (2012). *Descomposición para problemas de programación lineal multidivisionales* [Tesis de maestría, Universidad de Guanajuato, Guanajuato, México].
- Romero, E., Barrio, I., & Belanche, L. (s.f.). Incremental and decremental learning for linear support vector machines.
- Roth, D. (2016, octubre). Multiclass classification. <https://www.cis.upenn.edu/~danroth/Teaching/CS446-17/LectureNotesNew/multiclass/main.pdf>
- Sahu, C. K., & Sharma, M. (2023, febrero). Hinge loss in support vector machines.
- Shah, R. S. (2007). *Support vector machines for classification and regression* [Tesis de maestría, McGill University].
- Shalev-Shwartz, S., & Ben-David, S. (2014). *Understanding machine learning: From theory to algorithms*. Cambridge University Press. <http://www.cs.huji.ac.il/~shais/UnderstandingMachineLearning>
- Smola, A. J., & Schölkopf, B. (2004). A tutorial on support vector regression. *Statistics and Computing*, 14(3), 199-222. <https://doi.org/10.1023/B:STCO.0000035301.49549.88>

- Sontag, D. (s.f.). Support vector machines (SVMs) [Adaptado de L. Zettlemoyer, V. Gogate y C. Guestrin].
- Sun, L., Wei, X., & Liu, Y. (2020). Poisoning attack and defense: A survey. *The Computer Journal*, 63(5), 1-13. <https://doi.org/10.1093/comjnl/bxz141>
- University, C. M. (2020). Support Vector Machines (SVM).
- Vapnik, V. (1999). *The nature of statistical learning theory* (2nd). Springer. <https://doi.org/10.1007/978-1-4757-3264-1>
- Weerasinghe, S., Alpcan, T., et al. (2021). Defending Support Vector Machines Against Data Poisoning Attacks. *IEEE Transactions on Information Forensics and Security*, 16, 2566-2578.
- Weston, J., & Watkins, C. (1999, marzo). *Multiclass support vector machines* (inf. téc.). Department of Computer Science, Egham, Surrey TW EX, England. <https://www.researchgate.net/publication/221997121>
- Wikipedia. (s.f.). Aprendizaje automático. https://es.wikipedia.org/wiki/Aprendizaje_automatiko
- Xiao, H., Biggio, B., et al. (2015). Support vector machines under adversarial label contamination. *Neurocomputing*, 160, 53-62.
- Xiao, H., Biggio, B., Brown, G., Fumera, G., Eckert, C., & Roli, F. (2018). Is feature selection secure against training data poisoning? <https://arxiv.org/abs/1804.07933>
- Zhang, Z., & Wang, H. (2020). Poisoning attacks and defenses in machine learning: A survey. *Journal of Artificial Intelligence Research*, 67, 105-135. <https://doi.org/10.1613/jair.1.11744>
- Zhou, D., & Xu, C. (2017). Attacks and defenses for machine learning in security. *Proceedings of the 2017 IEEE Symposium on Security and Privacy*. <https://doi.org/10.1109/SP.2017.33>